

Machine Intelligence

2. Constraint Satisfaction Problems

What to Do When Your Problem is to Satisfy All These Constraints.

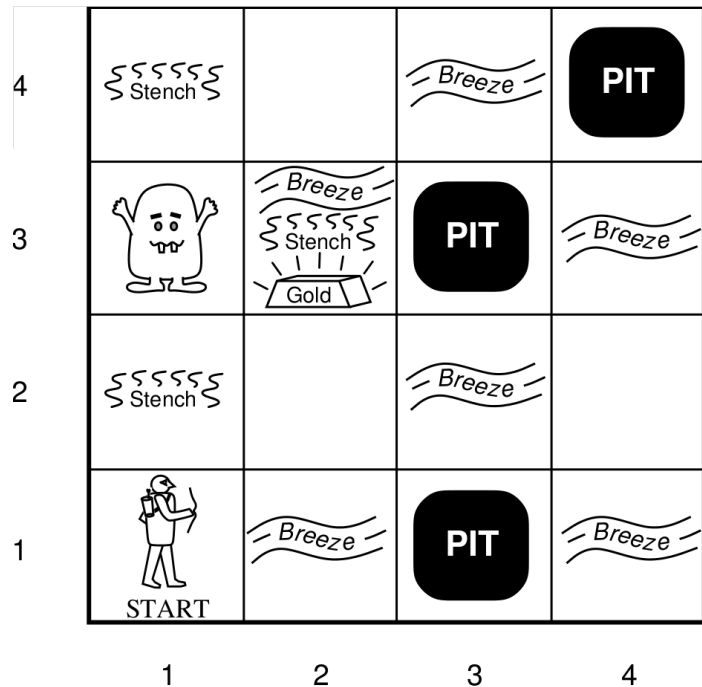
Álvaro Torralba



AALBORG UNIVERSITET

Fall 2024

The Wumpus World



- **Actions:** *GoForward*, *TurnRight* (by 90°), *TurnLeft* (by 90°), *Grab* object in current cell, *Shoot* arrow in direction you're facing (you got exactly one arrow), *Leave cave* if you're in cell [1,1].

→ **Fall down Pit, meet live Wumpus: Game Over.**

- **Initial knowledge:** You're in cell [1,1] facing east. There's a Wumpus, and there's gold.
- **Goal:** Have the gold and be outside the cave.

Percepts: [*Stench*, *Breeze*, *Glitter*, *Bump*, *Scream*]

- Cell adjacent (i.e. north, south, west, east) to Wumpus: *Stench* (else: *None*).
- Cell adjacent to Pit: *Breeze* (else: *None*).
- Cell that contains gold: *Glitter* (else: *None*).
- You walk into a wall: *Bump* (else: *None*).
- Wumpus shot by arrow: *Scream* (else: *None*).

Reasoning in the Wumpus World

A: Agent, **V:** Visited, **OK:** Safe, **P:** Pit, **W:** Wumpus, **B:** Breeze, **S:** Stench, **G:** Gold

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
A			
OK	OK		

(1) Initial state

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK	P?		
1,1	2,1	3,1	4,1
V	A	P?	
OK	B	OK	

(2) One step to right

1,4	2,4	3,4	4,4
1,3	W!	3,3	4,3
1,2	A	3,2	4,2
S	OK	OK	
1,1	2,1	3,1	4,1
V	B	P!	
OK	V	OK	

(3) Back, and up to [1,2]

→ **The Wumpus is in [1,3]! How do we know?** Because in [2,1] we perceived no Stench, the Stench in [1,2] can only come from [1,3].

→ **There's a Pit in [3,1]! How do we know?** Because in [1,2] we perceived no Breeze, the Breeze in [2,1] can only come from [3,1].

When you have eliminated the impossible, whatever remains, however improbable, must be the truth (Sherlock Holmes)

Propositional Logic: Syntax

Atomic Propositions

Boolean variables are now seen as **atomic propositions**.

Constraints	Logic
$A = \text{true}$	a
$A = \text{false}$	$\neg a$

Propositions (Formulas)

Using **logical connectives** more complex propositions are constructed:

$\neg p$	not p
$(p \wedge q)$	p and q
$(p \vee q)$	p or q
$(p \rightarrow q)$	p implies q

Example: “If it rains I’ll take my umbrella, or I’ll stay home”

$$\text{rains} \rightarrow (\text{umbrella} \vee \text{home})$$

Propositional Logic: Semantics

An **interpretation** π for a set of atomic propositions is an assignment of a truth value to each (= possible world when atomic propositions seen as Boolean variables)

→ **An interpretation defines a truth value for all propositions/formulas.**

<i>rains</i>	<i>umbrella</i>	<i>home</i>	$rains \rightarrow (umbrella \vee home)$
<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>
<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>false</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>
<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

How many interpretations are there in this example? 8

A **model** is an interpretation I in which the formula is true

→ **A model is a possible world that satisfies the constraints.**

How many models are there in this example? 7

A formula φ is **satisfiable** if it has a model (there exists I that satisfies φ).

Deduction

Knowledge Base

A **Knowledge Base (KB)** is a set of formulas that describe the agent's knowledge.

→ **Knowledge Base = set of formulas, interpreted as a conjunction.**

Deduction

deriving of a conclusion by reasoning

Remember (slide 3)? Does our knowledge entail a definite Wumpus position?

→ **We don't know everything; what can we conclude from the things we do know?**

Logical consequence (entailment)

A formula φ is a **logical consequence** of a knowledge base KB , if every model of KB is a model of φ . Written:

$$KB \models \varphi$$

(whenever KB is true, then I can conclude that φ also is true).

Example: $KB = \{man \rightarrow mortal, man\}$. Then $KB \models mortal$

Proof by Contradiction

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1	2,1	3,1	4,1
☐ A	OK		
OK			

Question!

Suppose that there exists an interpretation I in $M(KB)$ where the Wumpus is not at cell $(2, 2)$. Can we conclude the cell $(2, 2)$ is free?

(A): Yes

(B): No

No, it could not be there, but that does not mean that it is not there!

Contradiction Theorem. $KB \models \varphi$ if and only if $KB \cup \{\neg\varphi\}$ is unsatisfiable.

→ **Entailment can be tested via satisfiability.**

To deduce “Cell $(2,2)$ is free”, assume the opposite “ $\neg(\text{cell}(2,2) \text{ is free})$ ” look for an interpretation. If none exists, you can conclude the opposite of your assumption.

Does there exist an interpretation that agrees with all formulas in our KB?

Does there exist a variable assignment that agrees with all constraints? CSP!

In this course, we don't cover algorithms for testing satisfiability. But the principles are very similar to what we cover in Today's lecture.

A (Constraint Satisfaction) Problem



→ Who's going to play against who, when and where?

Constraint Satisfaction Problems

What is a constraint?

A **constraint** is a condition that every solution must satisfy.

What is a constraint satisfaction problem?

Given:

- A set of **variables**, each associated with its **domain**.
- A set of constraints over these variables.

Find:

- An **assignment** of variables to values (from the respective domains), so that every constraint is satisfied.

A Constraint Satisfaction Problem

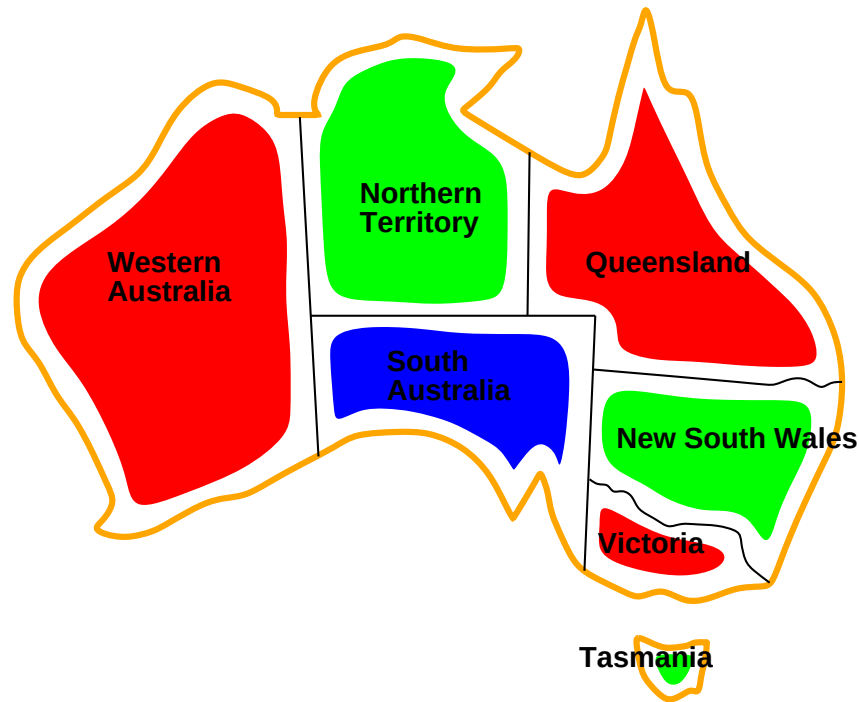
→ Problem: SuDoKu.

2	5	8	7	3	6	9	4	1
6	1	9	8	2	4	3	5	7
4	3	7	9	1	5	2	6	8
3	9	5	2	7	1	4	8	6
7	6	2	4	9	8	1	3	5
8	4	1	6	5	3	7	2	9
1	8	4	3	6	9	5	7	2
5	7	6	1	4	2	8	9	3
9	2	3	5	8	7	6	1	4

- **Variables:** Content of each cell.
- **Domains:** Numbers 1, ..., 9.
- **Constraints:** Each number only once in each row, column, block.

Another Constraint Satisfaction Problem

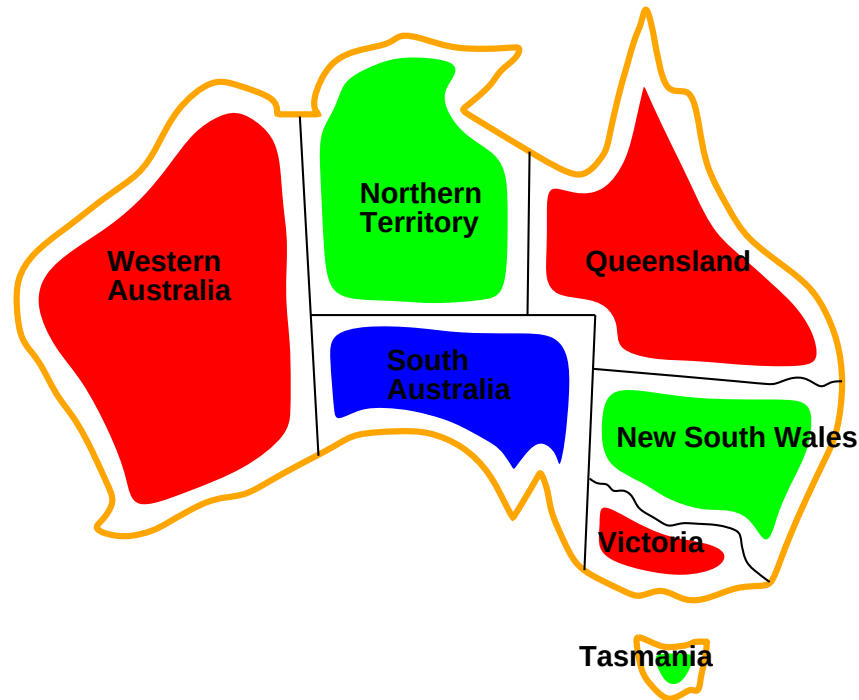
→ (Our Main Illustrative) Problem: Coloring Australia.



- **Variables:** WA, NT, SA, Q, NSW, V, T.
- **Domains:** red, green, blue.
- **Constraints:** Adjacent states must have different colors.

Another Constraint Satisfaction Problem

→ **Problem: Graph Coloring. NP-hard for $k = 3$.**



- **Variables:** Vertices in a graph.
- **Domains:** k different colors.
- **Constraints:** Adjacent vertices must have different colors.

Bundesliga Constraints

Variables: $v_{A\text{vs.}B}$ where A and B are teams, with domain $\{1, \dots, 34\}$: For each match, the (ID of the) “Spieltag” where it is scheduled.



(Some) Constraints:

- For all A, B : $v_{A\text{vs.}B} \leq 17 < v_{B\text{vs.}A}$ or $v_{B\text{vs.}A} \leq 17 < v_{A\text{vs.}B}$ (each pairing exactly once in each half-season).
- For all A, B, C, D where $\{A, B\} \cap \{C, D\} \neq \emptyset$: $v_{A\text{vs.}B} \neq v_{C\text{vs.}D}$ (each team only one match per day).
- For all A, B, D : $v_{A\text{vs.}B} + 1 \neq v_{A\text{vs.}D}$ (each team alternates between home matches and away matches).
- ...

How to Solve the Bundesliga Constraints?

Pre-study attempts:

- ① 306 nested for-loops (for each of the 306 matches), each ranging from 1 to 306. Within the innermost loop, test whether the current values are (a) a permutation and, if so, (b) a legal Bundesliga schedule.
→ **Estimated runtime: End of this universe, and the next ca. 10^{1000} ones after it ...**
- ② Directly enumerate all permutations of the numbers $1, \dots, 306$, test for each whether it's a legal Bundesliga schedule.
→ **Estimated runtime: Maybe only the time span of 10^{500} universes.**
- ③ View this as variables/constraints and use backtracking (**This Chapter**).
→ **Executed runtime: About 1 minute.**

How do they actually do it? Modern computers and CSP methods: fractions of a second.
19th (20th/21st?) century: Combinatorics and manual work.

Some Applications

Traveling Tournament Problem



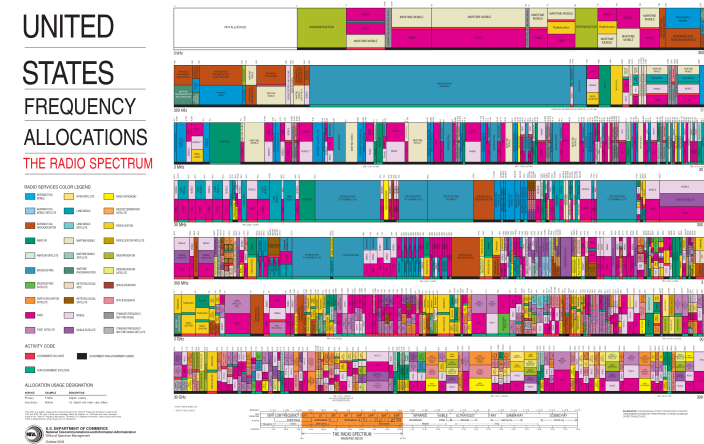
Scheduling



Timetabling



Radio Frequency Assignment



Features and Variables

Describing the world (environment) by features (variables)

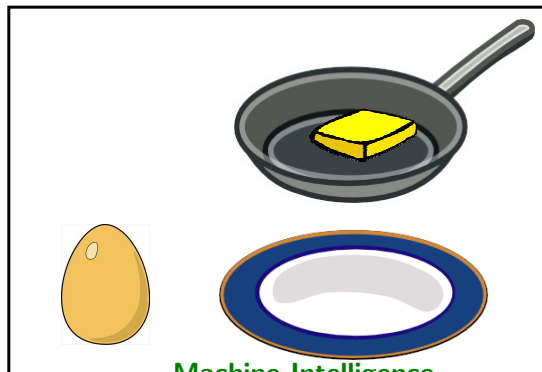
Variable	Domain
<i>egg</i>	$\{whole, broken\}$
<i>butter_in</i>	$\{pan, plate, table\}$
<i>egg_in</i>	$\{pan, plate, table\}$

- We will be mostly concerned with (algebraic) variables that have a *finite* domain.
- Numerical variables can be discretized and/or approximated:

$$\begin{aligned} \{1, 2, 3, \dots\} &\mapsto \{1, 2, 3, 4, 5, > 5\} \\ [-25, 40] &\mapsto \{-25, -24, \dots, -1, 0, 1, \dots, 40\} \end{aligned}$$

From variables to possible worlds

A **possible world** for a set of variables is an assignment of a value to each variable.
 One out of $2 \cdot 3 \cdot 3 = 18$ possible worlds:



egg=whole
butter_in=pan
egg_in=table

Constraint Satisfaction Problems

What is a constraint?

A **constraint** is a condition that every solution must satisfy.

What is a constraint satisfaction problem?

A **(binary) CSP** is a triple $\gamma = (V, D, C)$ where:

- A set of **variables**, each associated with its **domain**:
- $V = \{v_1, \dots, v_n\}$ is a finite set of **variables**.
- $D = \{D_{v_1}, \dots, D_{v_n}\}$ is a corresponding set of finite **domains**.
- C = a set of constraints over these variables.

Solution:

- An **assignment** of variables to values (from the respective domains), so that every constraint is satisfied.

Other tasks:

- Determine the number of models of the constraints
- Find an *optimal* model (given also a value function on possible worlds).
- ...

Example: Schedule

Variables:

<i>Teacher_MI</i>	$\{PD, MJ, TDN\}$
<i>Time_MI</i>	$\{Mo_m, Mo_a, \dots, Fr_m, Fr_a\}$
<i>Room_MI</i>	$\{0.2.12, 0.2.13, 0.2.90\}$
<i>Teacher_AD</i>	$\{PD, MJ, TDN\}$
<i>Time_AD</i>	$\{Mo_m, Mo_a, \dots, Fr_m, Fr_a\}$
<i>Room_AD</i>	$\{0.2.12, 0.2.13, 0.2.90\}$

One possible world:

Mo	Tue	Wed	Thu	Fr
	MI, TDN, 0.2.13			
			AD, PD, 0.2.90	

Teacher_MI=TDN
Time_MI=Tue_m
Room_MI=0.2.13
Teacher_AD=PD
Time_AD=Thu_a
Room_AD=0.2.90

Constraints

A **constraint** is a condition on the values of variables in a possible world.

Extensional Constraint Specification

Explicitly list all allowed (or disallowed) combination of values for a subset of variables:

<i>Teacher_MI</i>	<i>Time_MI</i>	<i>Room_MI</i>	<i>Teacher_AD</i>	<i>Time_AD</i>	<i>Room_AD</i>
PD	<i>Mo_m</i>	0.2.12	PD	<i>Mo_a</i>	0.2.12
PD	<i>Mo_m</i>	0.2.12	PD	<i>Mo_a</i>	0.2.13
...	...	...	...	...	...

→Special cases:

- **Unary Constraints:** single column (restrict values of a variable to a sub-set of variable)
- **Binary Constraints:** two columns (variables u and v), $C_{u,v}$ restricts the values that the two variables can have together.

Intensional Constraint Specification

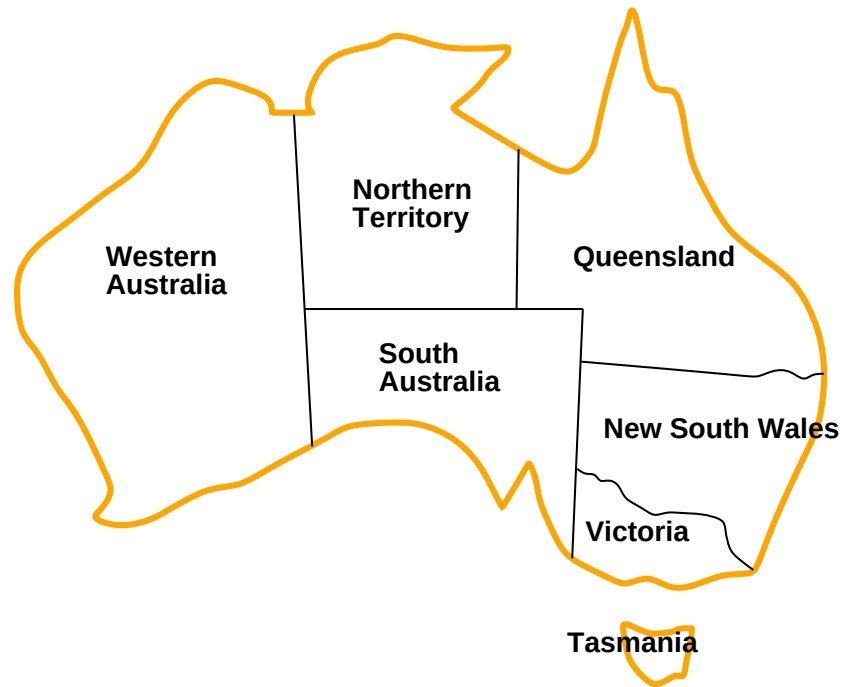
Use logical expressions:

$$Teacher_AD = Teacher_MI \rightarrow Time_AD \neq Time_MI$$

$$Time_AD = Time_MI \rightarrow Room_AD \neq Room_MI$$

→Can be transformed to an extensional constraints (but sometimes it may require exponential time/space)

Example: Coloring Australia



- **Variables:** $V = \{WA, NT, SA, Q, NSW, V, T\}$.
- **Domains:** For all $v \in V$: $D_v = \{red, green, blue\} =: D$.
 → If all variables have the same domain, abusing notation we will write D to denote that “global” domain.
- **Constraints:** C_{uv} for adjacent states u and v , with $C_{uv} = "u \neq v"$, i.e., $C_{uv} = \{(d, d') \in D \times D \mid d \neq d'\}$.

Example: SuDoKu

2	5			3		9		1
	1				4			
4		7				2		8
		5	2					
				9	8	1		
	4				3			
			3	6			7	2
	7							3
9		3				6		4

- **Variables:** $V = \{v_{ij} \mid 1 \leq i, j \leq 9\}$: v_{ij} = cell row i column j .
- **Domains:** For all $v \in V$: $D_v = D = \{1, \dots, 9\}$.
- **Unary Constraints:** $C_{v_{ij}} = \{d\}$ if cell i, j is pre-filled with d .
- **Binary Constraints:** $C_{v_{ij}v_{i'j'}}$ = " $v_{ij} \neq v_{i'j'}$ ", i.e.,
 $C_{v_{ij}v_{i'j'}} = \{(d, d') \in D \times D \mid d \neq d'\}$, for: $i = i'$ (same row), or $j = j'$ (same column), or $(\lceil \frac{i}{3} \rceil, \lceil \frac{j}{3} \rceil) = (\lceil \frac{i'}{3} \rceil, \lceil \frac{j'}{3} \rceil)$ (same block).

CSP and the Model-and-Solve Paradigm

(some new constraint-reasoning problem)



describe problem as a constraint network $\mapsto$ use off-the-shelf CSP solver



(its solution)

- Constraint networks=generic language to describe this kind of problem.
- CSP solvers=generic algorithms solving such problems.
- The next time you play SuDoKu, just write the game down in CSP format and use an off-the-shelf solver.

Examples:

- <https://www.minizinc.org/index.html>
- https://developers.google.com/optimization/cp/cp_solver
- <https://www.ibm.com/products/ilog-cplex-optimization-studio/cplex-cp-optimizer>

Assignments and Consistency

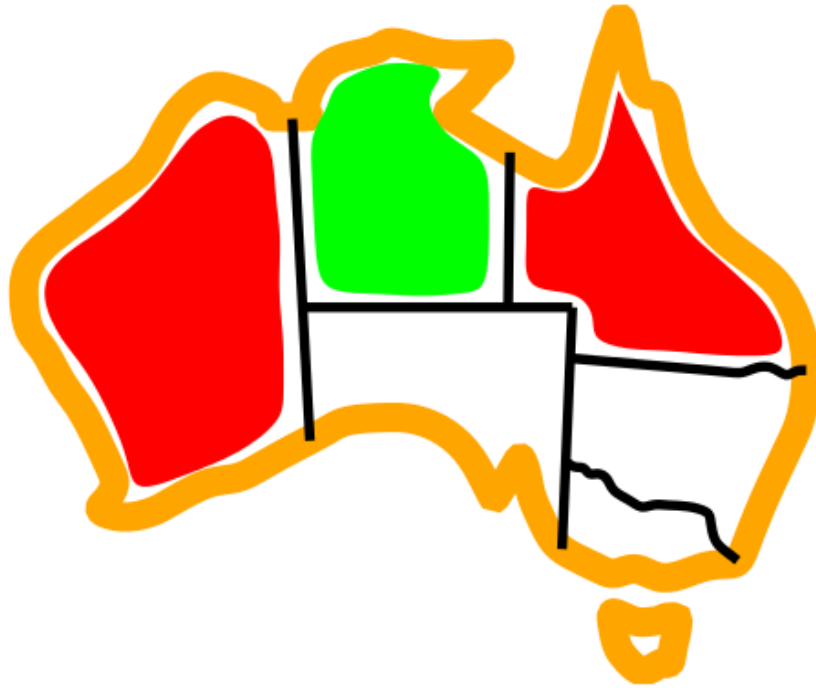
Definition (Assignment). Let $\gamma = (V, D, C)$ be a constraint network. A **partial assignment** is a function $a : V' \mapsto \bigcup_{v \in V} D_v$ where $V' \subseteq V$ and $a(v) \in D_v$ for all $v \in V'$. If $V' = V$, then a is a **total assignment**, or **assignment** in short.

→ A partial assignment assigns some variables to values from their respective domains. A total assignment is defined on all variables.

Definition (Consistency). Let $\gamma = (V, D, C)$ be a constraint network, and let a be a partial assignment. We say that a is **inconsistent** if there exist variables $u, v \in V$ on which a is defined, with $C_{uv} \in C$ and $(a(u), a(v)) \notin C_{uv}$. In that case, a **violates** the constraint C_{uv} . We say that a is **consistent** if it is not inconsistent.

→ Partial assignment inconsistent = “already violates a constraint”. (Trivially consistent: The empty assignment.)

Example: Coloring Australia



Is this partial assignment
consistent? Yes.

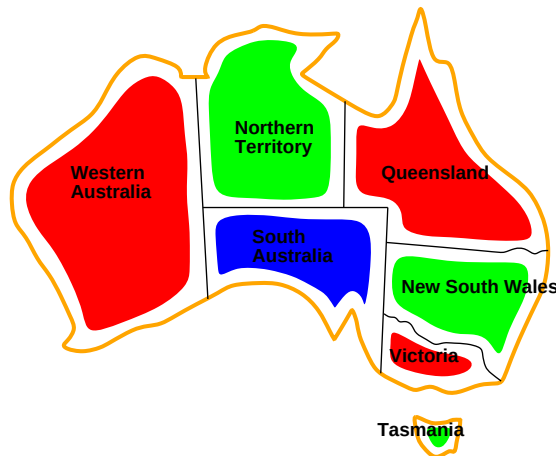


Is this partial assignment
consistent? No.

Solutions

Definition (Solution). Let $\gamma = (V, D, C)$ be a constraint network. If a is a **total consistent assignment**, then a is a **solution** for γ . If a solution to γ exists, then γ is **solvable**; otherwise, γ is **inconsistent**.

Example “Coloring Australia”:



- **Variables:** $V = \{WA, NT, SA, Q, NSW, V, T\}$.
- **Domains:** All $v \in V$: $D_v = D = \{red, green, blue\}$.
- **Constraints:** C_{uv} for adjacent states u and v , with $C_{uv} = \{(d, d') \in D \times D \mid d \neq d'\}$.
- **Solution:** $\{WA = red, NT = green, SA = blue, Q = red, NSW = green, V = red, T = green\}$.

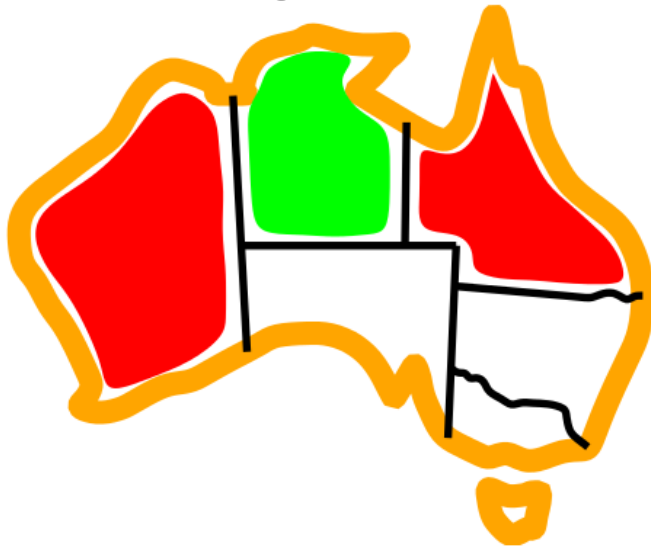
→ Note: This is not the only solution. E.g., we can permute the colors, and Tasmania can be assigned an arbitrary color.

Consistency vs. Extensibility

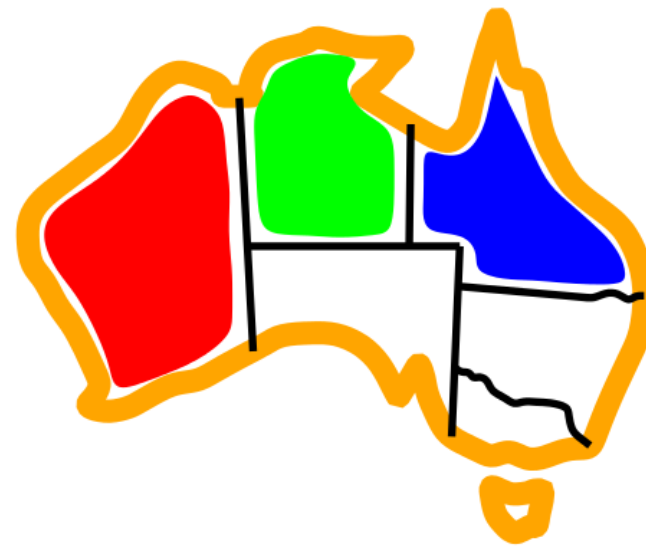
Definition. Let $\gamma = (V, D, C)$ be a constraint network, and let a be a partial assignment. We say that a can be **extended to a solution** if there exists **a solution a' that agrees with a on the variables where a is defined.**

→ a can be extended to a solution $\implies a$ consistent. But not vice versa:

Example “Coloring Australia”:



Can this partial assignment be extended to a solution? Yes.



Can this partial assignment be extended to a solution? No.

Consistency vs. Extensibility

Definition. Let $\gamma = (V, D, C)$ be a constraint network, and let a be a partial assignment. We say that a can be **extended to a solution** if there exists **a solution a' that agrees with a on the variables where a is defined.**

→ a can be extended to a solution $\implies a$ consistent. But not vice versa:

Example “4-Queens”:

	v_1	v_2	v_3	v_4
1	q			
2			q	
3				
4		q		

Can this partial assignment be extended to a solution? No.

CSP as State Space Problem

A CSP can be represented as a state space problem:

- States are all **partial assignments of values to variables that are consistent with the constraints**
- Actions applicable on state s : select some variable V not assigned a value in s , and let the neighbors of s be all states that assign a value to V (if any exist).
- The start state is the state that does not assign any values
- A goal state is a state that assigns values to all variables

Solving the CSP

- A solution to the state space problem is a path with a goal state at the end: a solution to the CSP problem
- To solve the state space problem need only be able to:
 - enumerate all partial assignments that assign a value to one more variable than s
 - check whether a partial assignment is consistent with the constraints
 (that is sufficient to implement the *get_neighbors* and *goal* functions needed in the generic search algorithm)

→ Contrary to **Chapter 9**, here we care only about the state reached, and not about the path

Naïve Backtracking

Call with input constraint network γ and the empty assignment a :

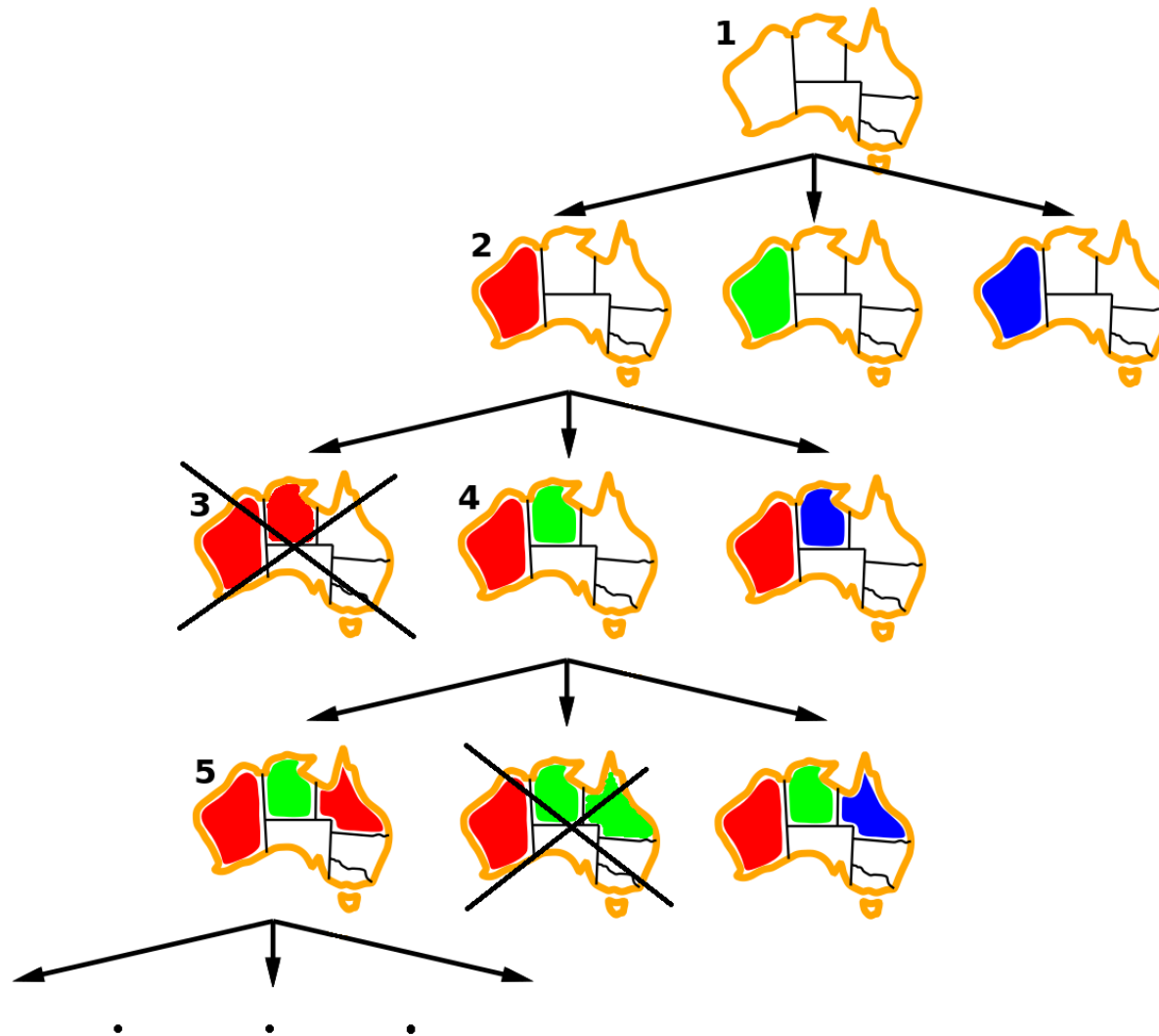
```

function NaïveBacktracking( $a$ ) returns a solution, or “inconsistent”
  if  $a$  is inconsistent then return “inconsistent”
  if  $a$  is a total assignment then return  $a$ 
  select some variable  $v$  for which  $a$  is not defined
  for each  $d \in D_v$  in some order do
     $a' := a \cup \{v = d\}$ 
     $a'' := \text{NaïveBacktracking}(a')$ 
    if  $a'' \neq$  “inconsistent” then return  $a''$ 
  return “inconsistent”
  
```

→ Backtracking=Recursively instantiate variables one-by-one, backing up out of a search branch if the current partial assignment is already inconsistent.

→ **Why is this better than enumerating, and solution-checking, all total assignments (cf. slide 15)?** If a partial assignment is already inconsistent, then backtracking does not enumerate any extensions thereof.

Example: Coloring Australia



Naïve backtracking, Pro and Contra

Pro:

- Naïve backtracking is **complete** (if there is a solution, backtracking will find it).
- Naïve backtracking is **extremely simple**.
- Despite this simplicity, it is **much more efficient than enumerating total assignments**.
 “fill cells one-by-one and stop already when an illegal row/column/block occurs” vs. **“fill all cells, then check whether it’s a solution”**.

	5	8	7		6	9	4	1
		9	8	1	4	3	5	7
4		7	9		5	2	6	8
3	9	5	2	7	1	4	8	6
7	6	2	4	9	8	1	3	5
8	4	1	6	5	3	7	2	9
1	8	4	3	6	9	5	7	2
5	7	6	1	4	2	8	9	3
9	2	3	5	8	7	6	1	4

Contra:

- Backtracking does not recognize *a* that cannot be extended to a solution, unless *a* is already inconsistent.
“don’t think at all, just fill in cells and see where you get to”. In the present example, we would not even try to see the issue, and instead just keep filling in values, e.g. trying to put 5 or 8 into the top left cell.

→ **“Human SuDoKu playing” = lots of inference!** (You want to minimize the number of failed attempts to keep track of on paper ...)

Inference: Basic Facts

Inference

Deducing additional constraints (unary or binary), that follow from the already known constraints, i.e., that are satisfied in all solutions.

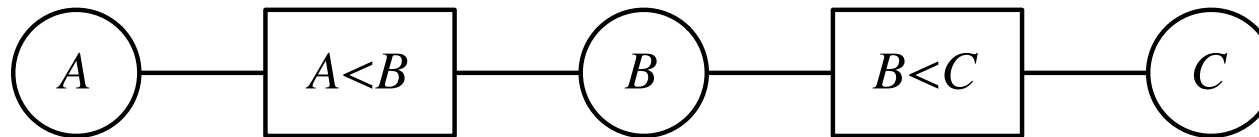
It's what you do all the time when playing SuDoKu:

	5	8	7		6	9	4	1
		9	8		4	3	5	7
4		7	9		5	2	6	8
3	9	5	2	7	1	4	8	6
7	6	2	4	9	8	1	3	5
8	4	1	6	5	3	7	2	9
1	8	4	3	6	9	5	7	2
5	7	6	1	4	2	8	9	3
9	2	3	5	8	7	6	1	4

→ **Formally: Replace γ by an equivalent and strictly tighter constraint network γ' .**

Networks could be made tighter in different ways (e.g. adding additional constraints). Here, we consider just reducing the domain of the variables.

Constraint Network



The **constraint network** for a CSP consists of

- One (oval) node for each variable X
- One (rectangular) node for each constraint c
- An (undirected) arc $\langle X, c \rangle$ between every constraint and every variable involved in the constraint

With each variable node X is associated a **(reduced) domain** D_X :

- Initially the domain of the variable
- Reduced by successively deleting values that cannot be part of a solution

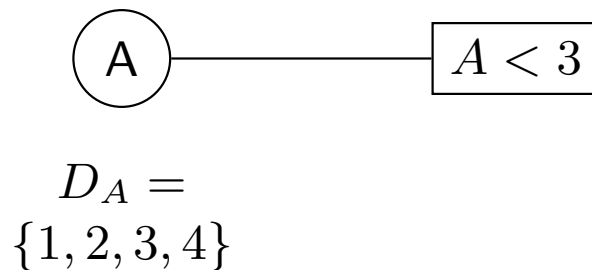
Arc Consistency: Definition

Definition (Arc Consistency). Let $\gamma = (V, D, C)$ be a constraint network.

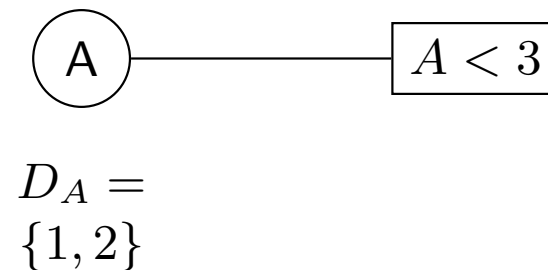
- (i) An arc $\langle v \in V, c \in C \rangle$ is **arc consistent** if for all $x \in D_v$ there exist values $y_1, \dots, y_k$ for the other variables involved in c , such that $v = x, v_1 = y_1, \dots, v_k = y_k$ is consistent with c .
- (ii) The network γ is **arc consistent** if all its arcs are.

→ Arc consistency = for every domain value and constraint, at least one value on the other side(s) of the constraint “works”.

Examples

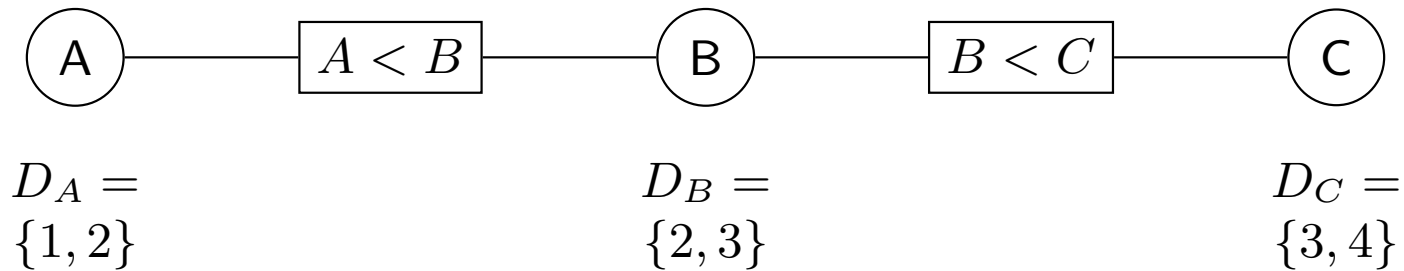


Arc consistent? No!

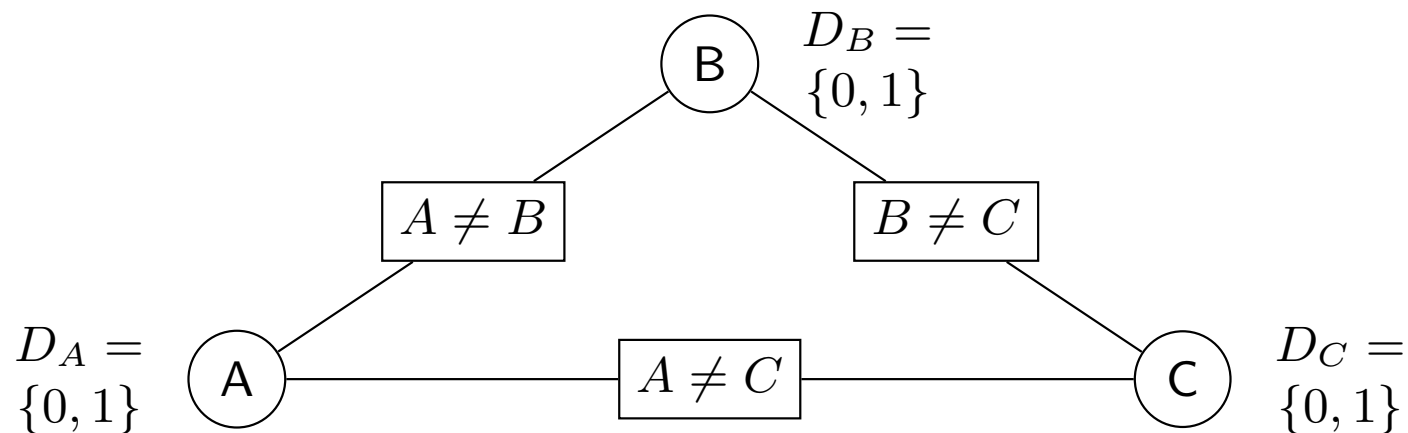


Arc consistent

Arc Consistency Examples



Is it arc consistent? Yes, even though not every combination of values from D_A, D_B, D_C is a solution!



Is it arc consistent? Yes, even though there exists no solution!

Enforcing Arc Consistency for *One* Pair of Variables

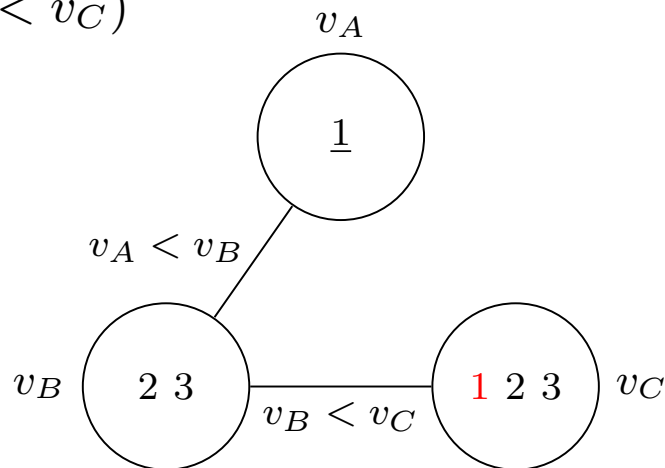
Algorithm enforcing consistency of u relative to binary constraint c :

```

function Revise( $\gamma, u, C_{uv}$ ) returns modified  $\gamma$ 
  for each  $d \in D_u$  do
    if there is no  $d' \in D_v$  with  $(d, d') \in C_{uv}$  then  $D_u := D_u \setminus \{d\}$ 
  return  $\gamma$ 
    
```

→ **Runtime, if k is maximal domain size:** $O(k^2)$, based on implementation where the test “ $(d, d') \in C_{uv}$?” is constant time.

Example: $\text{Revise}(\gamma, v_C, v_B < v_C)$



Enforcing Arc Consistency for *One* Pair of Variables

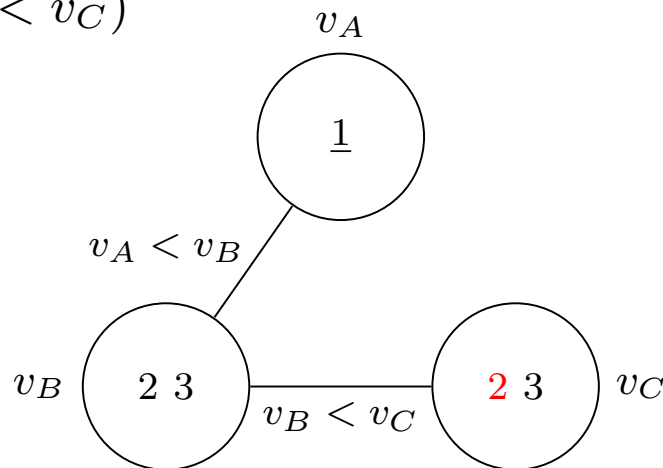
Algorithm enforcing consistency of u relative to binary constraint c :

```

function Revise( $\gamma, u, C_{uv}$ ) returns modified  $\gamma$ 
  for each  $d \in D_u$  do
    if there is no  $d' \in D_v$  with  $(d, d') \in C_{uv}$  then  $D_u := D_u \setminus \{d\}$ 
  return  $\gamma$ 
    
```

→ **Runtime, if k is maximal domain size:** $O(k^2)$, based on implementation where the test “ $(d, d') \in C_{uv}$?” is constant time.

Example: $\text{Revise}(\gamma, v_C, v_B < v_C)$



Enforcing Arc Consistency for *One* Pair of Variables

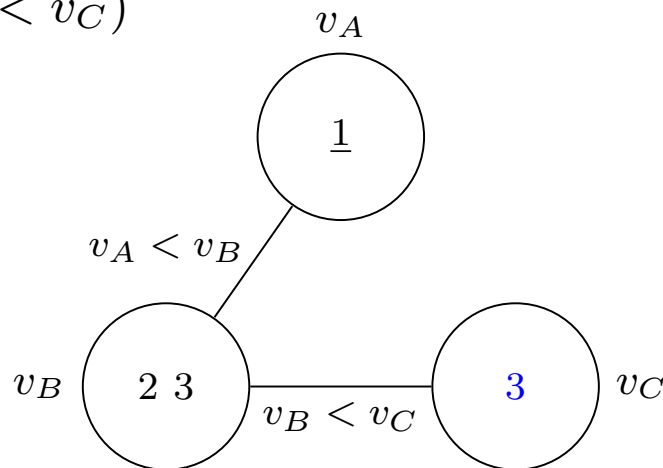
Algorithm enforcing consistency of u relative to binary constraint c :

```

function Revise( $\gamma, u, C_{uv}$ ) returns modified  $\gamma$ 
  for each  $d \in D_u$  do
    if there is no  $d' \in D_v$  with  $(d, d') \in C_{uv}$  then  $D_u := D_u \setminus \{d\}$ 
  return  $\gamma$ 
    
```

→ **Runtime, if k is maximal domain size:** $O(k^2)$, based on implementation where the test “ $(d, d') \in C_{uv}$?” is constant time.

Example: $\text{Revise}(\gamma, v_C, v_B < v_C)$

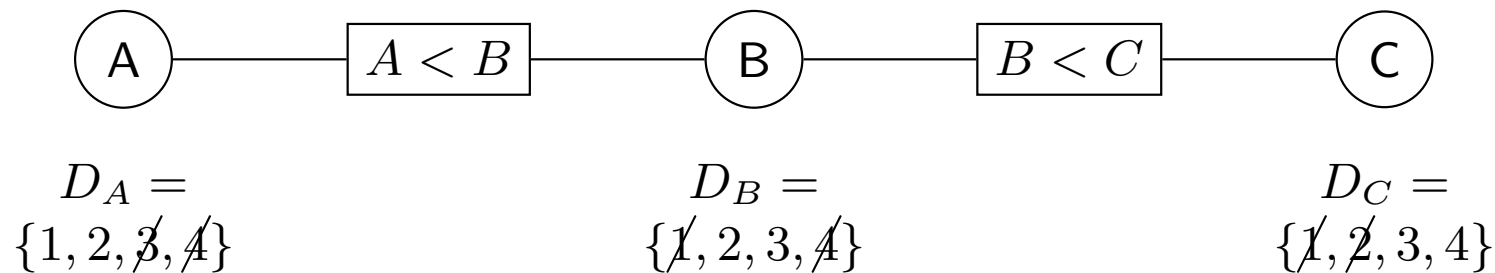


Generalized Arc Consistency Algorithm

Algorithm Outline

1. *To-do-arcs* = all arcs in constraint network // Potentially inconsistent arcs
2. **while** *To-do-arcs* $\neq \emptyset$
3. select and delete one arc $\langle X, c \rangle$ from *To-do-arcs*
4. make arc consistent by deleting values from D_X , if necessary
5. **if** values were deleted:
6. add all other arcs $\langle Z, c' \rangle$ ($c \neq c'$, $X \in \text{dom}(c')$) to *To-do-arcs*

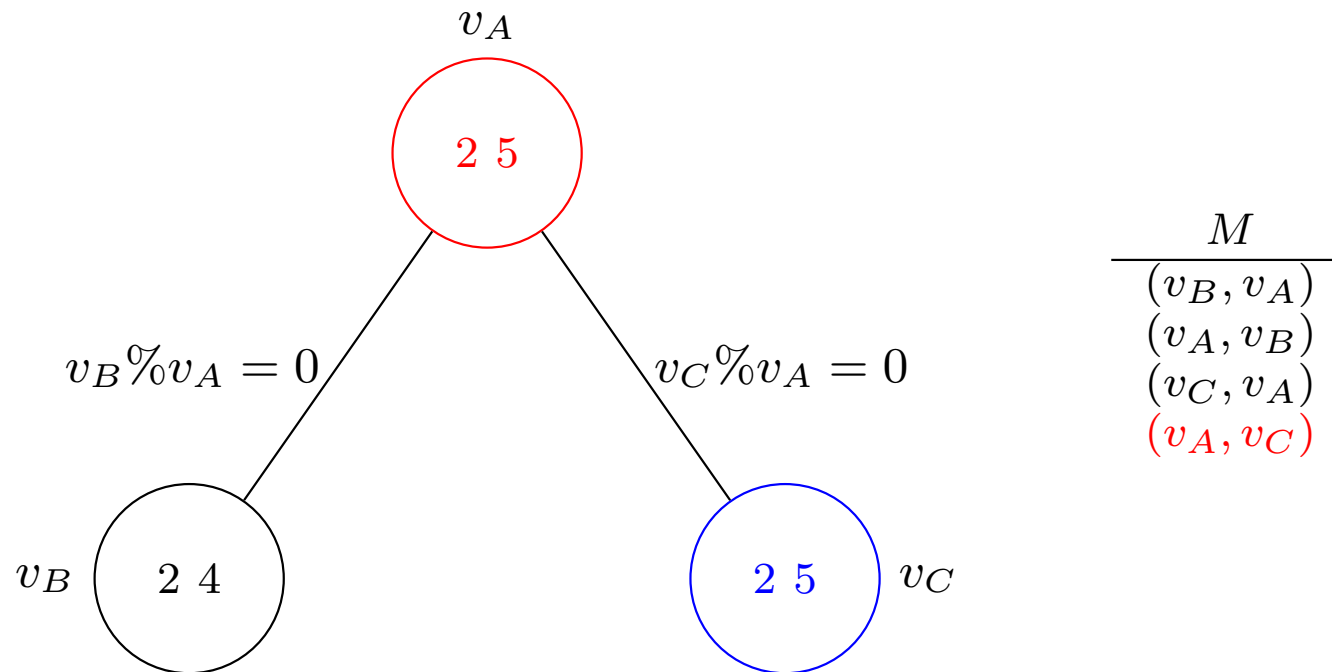
Example



- *To-do-arcs* = $\{\langle A, A < B \rangle, \langle B, A < B \rangle, \langle B, B < C \rangle, \langle C, B < C \rangle\}$
- *To-do-arcs* = $\{\langle B, A < B \rangle, \langle B, B < C \rangle, \langle C, B < C \rangle\}$
- *To-do-arcs* = $\{\langle B, B < C \rangle, \langle C, B < C \rangle\}$
- *To-do-arcs* = $\{\langle A, A < B \rangle, \langle C, B < C \rangle\}$
- *To-do-arcs* = $\{\langle C, B < C \rangle\}$

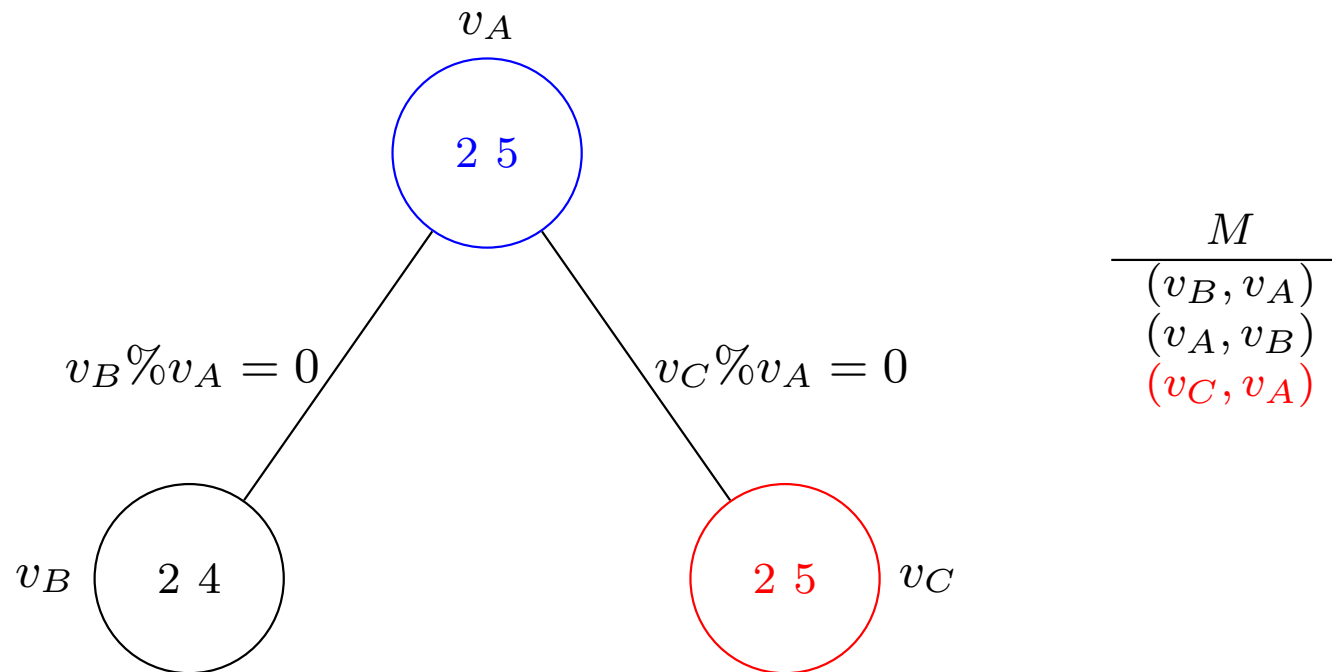
Arc Consistency Another Example

Example: $(y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)



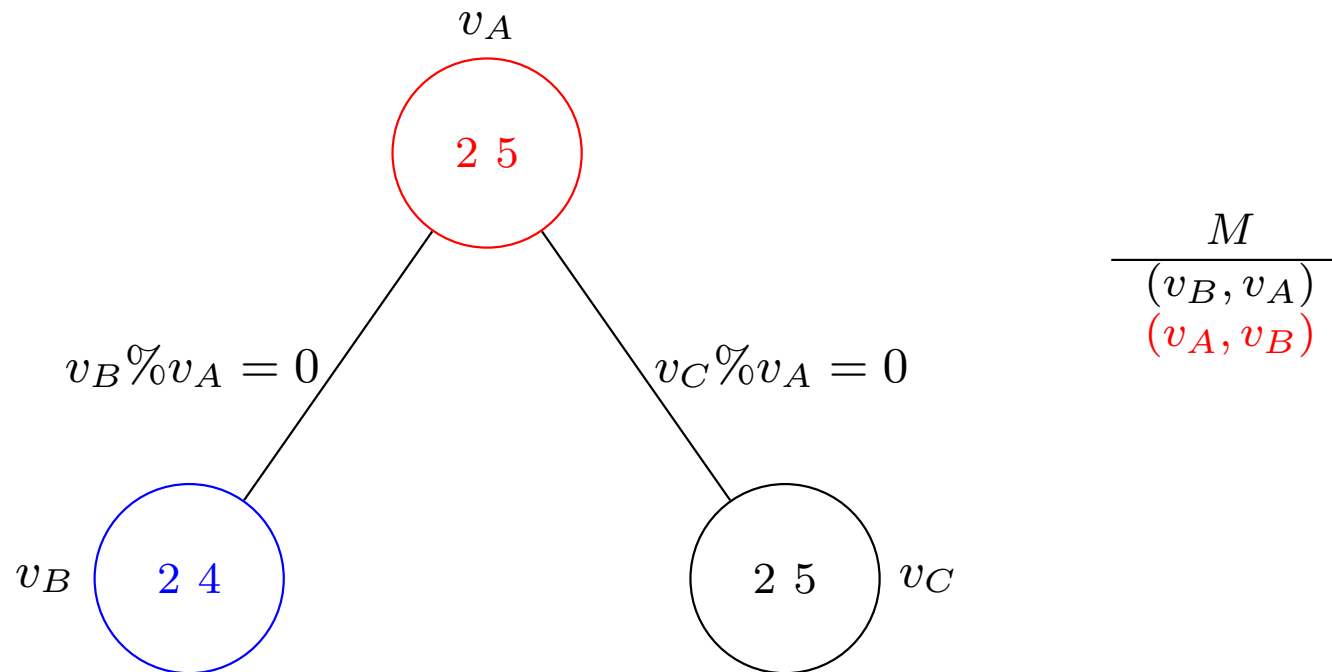
Arc Consistency Another Example

Example: $(y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)



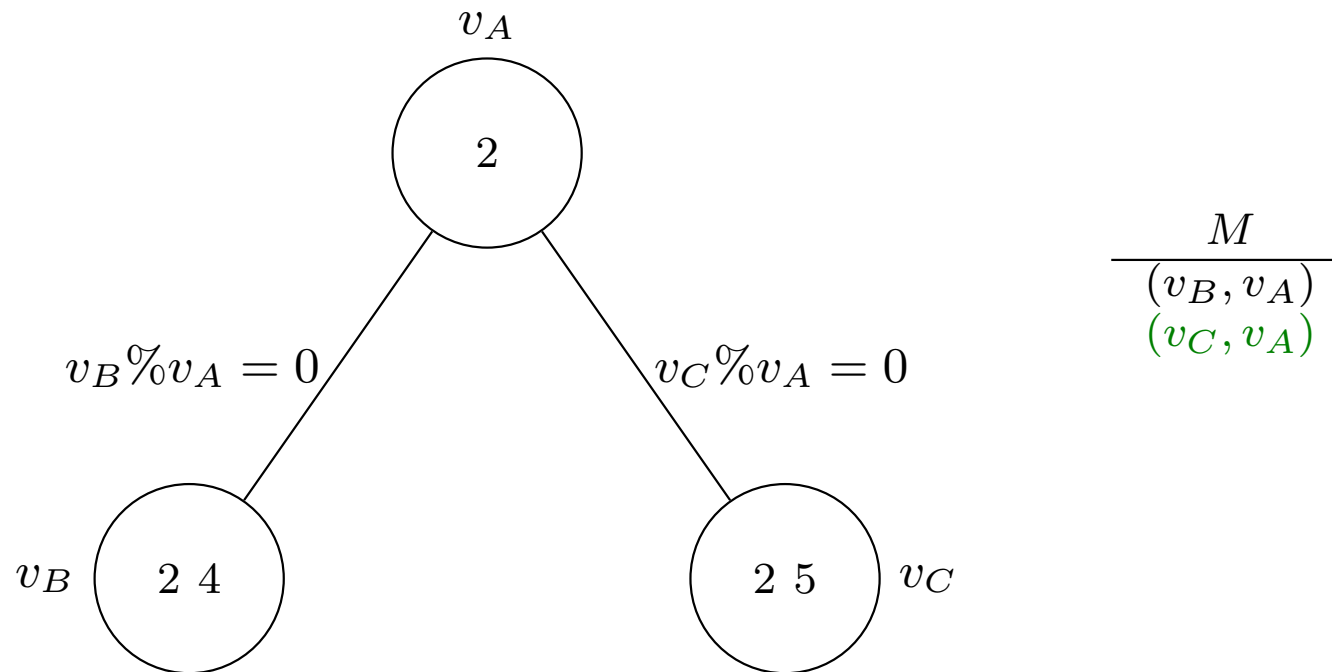
Arc Consistency Another Example

Example: ($y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)



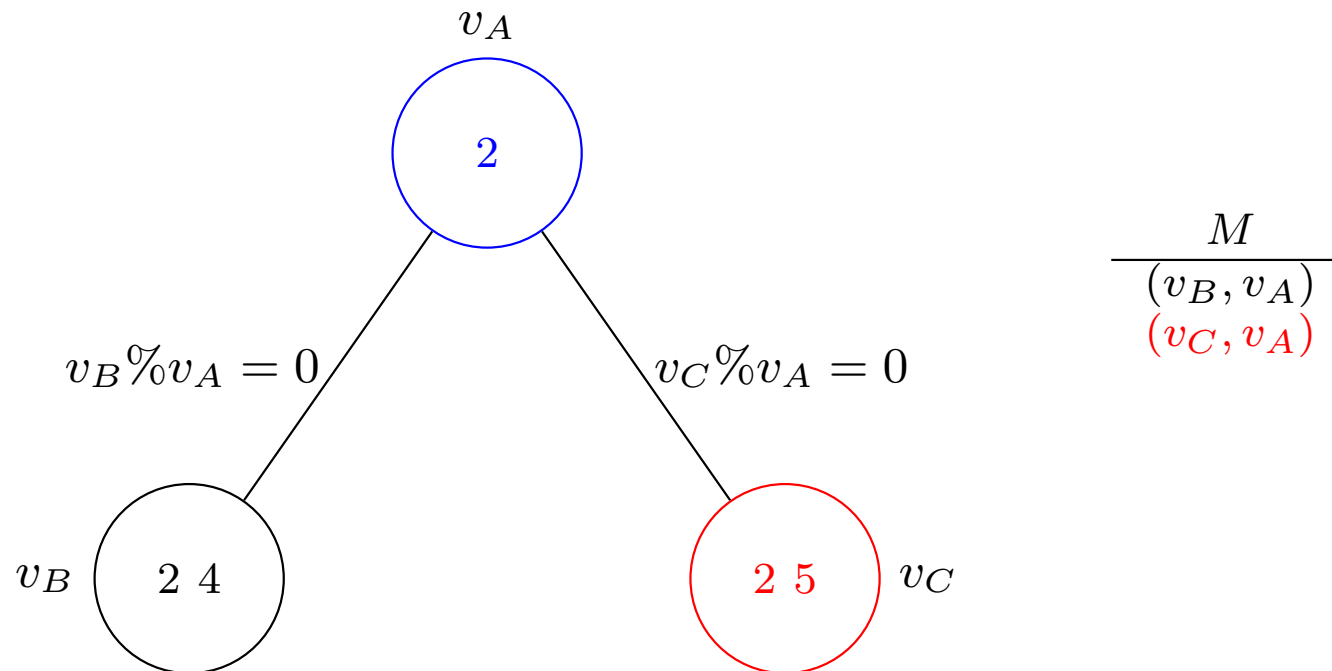
Arc Consistency Another Example

Example: $(y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)



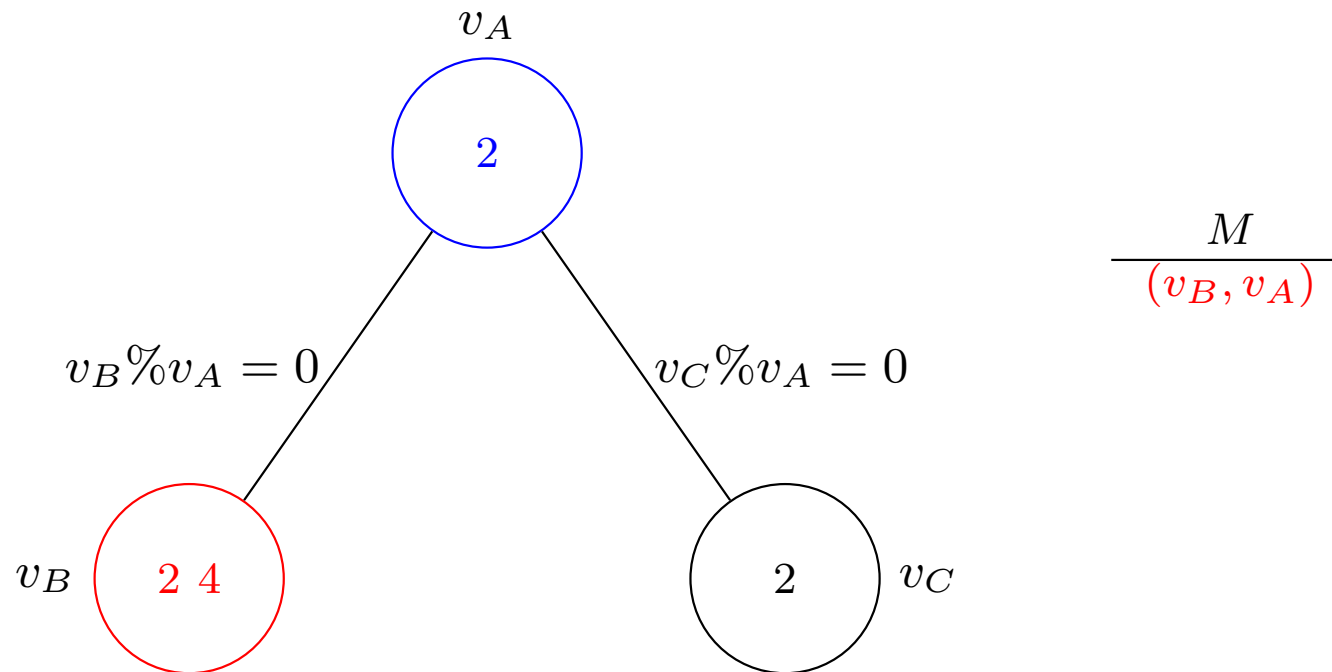
Arc Consistency Another Example

Example: ($y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)



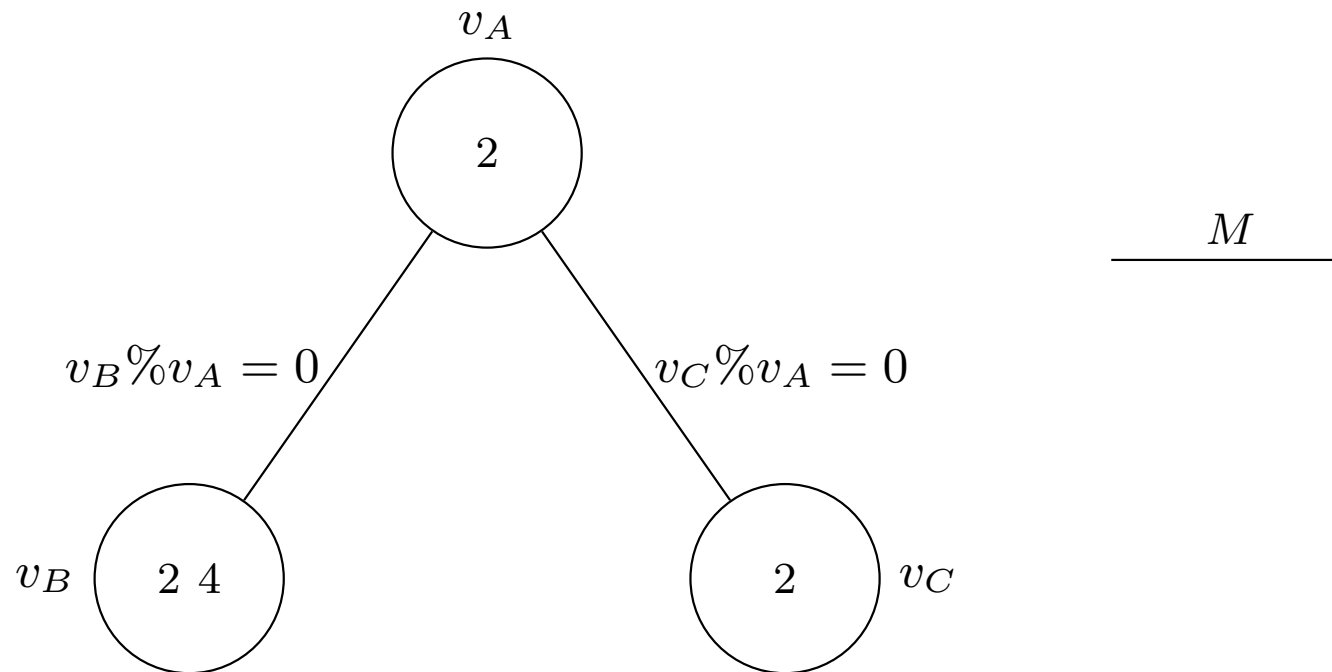
Arc Consistency Another Example

Example: ($y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)

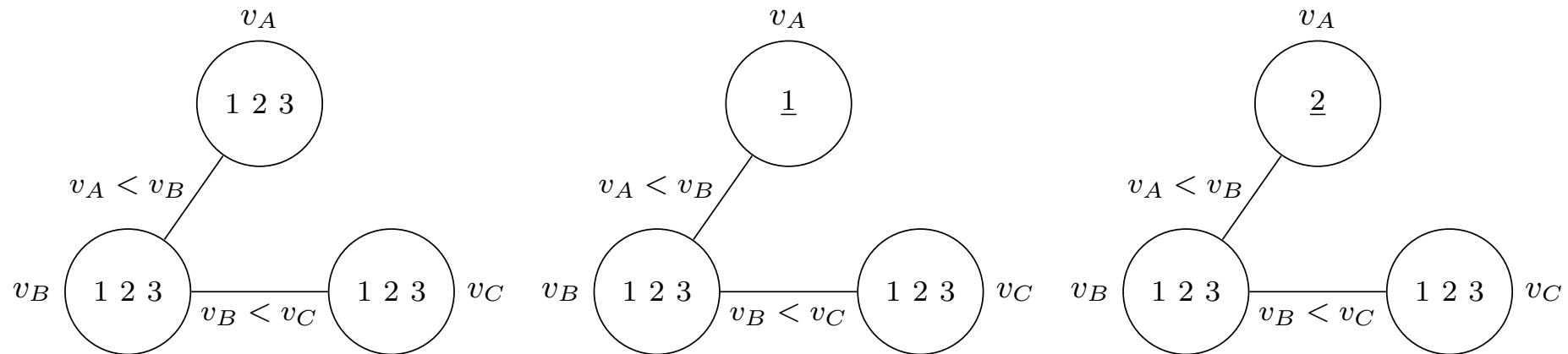


Arc Consistency Another Example

Example: ($y \% x = 0$: y modulo x is 0, i.e., y can be divided by x)



Questionnaire



Question!

Which inferences does enforcing arc consistency make, for each of these partial assignments?

- Left: $\text{Revise}(v_B, v_C)$ reduces D_{v_B} to $\{1, 2\}$, $\text{Revise}(v_B, v_A)$ then reduces it to $\{2\}$. From here, $\text{Revise}(v_A, v_B)$ and $\text{Revise}(v_C, v_B)$ reduce each domain to a singleton. Thus **enforcing arc consistency solves this network**.
- Middle: Same. (Special case of Left).
- Right: $\text{Revise}(v_B, v_C)$, $\text{Revise}(v_B, v_A)$ reduces D_{v_B} to $\emptyset$. Thus **enforcing arc consistency determines that this partial assignment cannot be extended to a solution**. cf. slide ??.)

How to Use Inference?

Inference as a pre-process:

- **Just once before search starts.**
- Little runtime overhead, little pruning power. Not considered here.

Inference during search:

- **At every recursive call of backtracking.**
- Strong pruning power, may have large runtime overhead.

Search vs. Inference

The more complex the inference, the *smaller* the number of search nodes, but the *larger* the runtime needed at each node.

Backtracking With Inference

```

function BacktrackingWithInference( $\gamma, a$ ) returns a solution, or “inconsistent”
  if  $a$  is inconsistent then return “inconsistent”
  if  $a$  is a total assignment then return  $a$ 

   $\gamma' :=$  a copy of  $\gamma$       /*  $\gamma' = (V, D', C')$  */
   $\gamma' :=$  Inference( $\gamma'$ )
  if exists  $v$  with  $D'_v = \emptyset$  then return “inconsistent”

  select some variable  $v$  for which  $a$  is not defined
  for each  $d \in$  copy of  $D'_v$  in some order do
     $a' := a \cup \{v = d\}; D'_v := \{d\}$       /* makes  $a$  explicit as a constraint */
     $a'' :=$  BacktrackingWithInference( $\gamma', a'$ )
    if  $a'' \neq$  “inconsistent” then return  $a''$ 
  return “inconsistent”
  
```

- **Inference()**: Any procedure delivering a (tighter) equivalent network (here, Generalized Arc Consistency initialized with constraints affected by the last variable assignment).
- Inference typically prunes domains; indicate unsolvability by $D'_v = \emptyset$.
- Partial assignment $\rightarrow$ unary constraints in γ' for call of inference in next recursion level.
- When backtracking out of a search branch, retract the inferred constraints: these were dependent on a , the search commitments so far.

Variable Elimination: Idea

- Arc Consistency: simplify problem by eliminating values
- **Variable Elimination**: simplify problem by eliminating variables

Variable Elimination operates on extensional (table) representations of constraints:
 → Each constraint can be represented as a (relational) table

Examples:

$A < B$:

A	B
1	2
1	3
1	4
2	3
2	4
3	4

$B < C$:

B	C
1	2
1	3
1	4
2	3
2	4
3	4

Algorithm requires two types of operators on tables: **projection** and **join**

Projection

Projection: **Remove some columns and eliminate duplicate rows**

Example:

Course	Year	Student	Grade
cs322	2008	fran	77
cs111	2009	billie	88
cs111	2009	jess	78
cs444	2008	fran	83
cs322	2009	jordan	92

$\pi_{\{Student, Year\}}$
 $\longrightarrow$

Student	Year
fran	2008
billie	2009
jess	2009
jordan	2009

Join

Given two tables r_1, r_2 for variable sets $vars_1, vars_2$. The **join** is the table $r_3 = r_1 \bowtie r_2$ for variables $vars_1 \cup vars_2$ that

- contains all tuples, which restricted to $vars_1$ are in r_1 , and restricted to $vars_2$ are in r_2 .

Example

Course	Year	Student	Grade		Course	Year	TA	
cs322	2008	fran	77	$\bowtie$	cs322	2008	yuki	$=$
cs111	2009	billie	88		cs111	2009	sam	
cs111	2009	jess	78		cs111	2009	chris	
cs444	2008	fran	83		cs322	2009	yuki	
cs322	2009	jordan	92					

Course	Year	Student	Grade	TA
cs322	2008	fran	77	yuki
cs111	2009	billie	88	sam
cs111	2009	jess	78	sam
cs111	2009	billie	88	chris
cs111	2009	jess	78	chris
cs322	2009	jordan	92	yuki

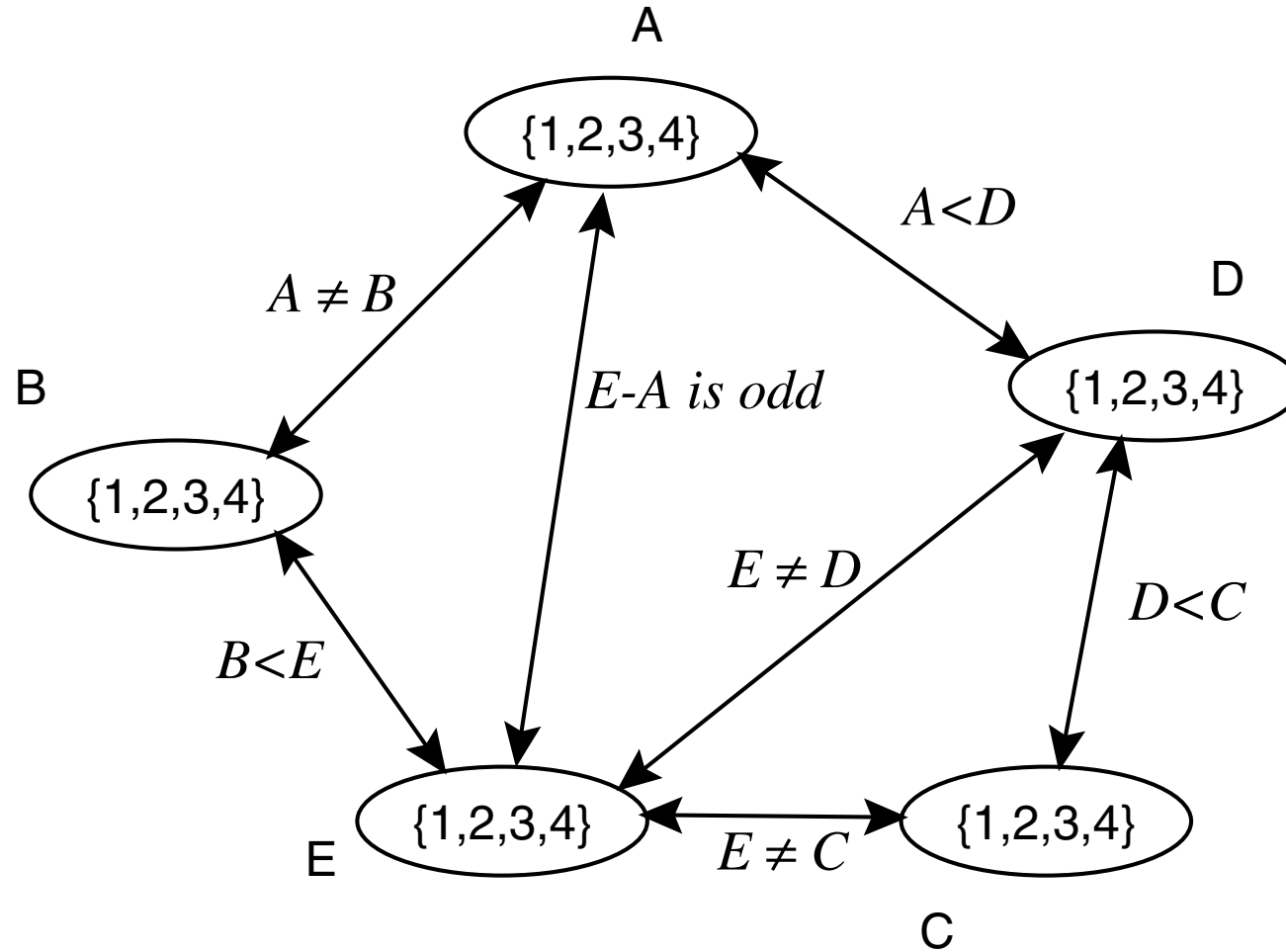
Variable Elimination Algorithm

Algorithm Outline

1. **Input:** C : set of constraints on variables $vars$
2. **while** C contains more than one element
3. select a variable $X \in vars$
4. delete X from $vars$
5. remove all constraints involving X from C and construct their join
6. **if** $vars$ is not empty
7. project the join onto the variables other than X
8. add the (projected) join to C

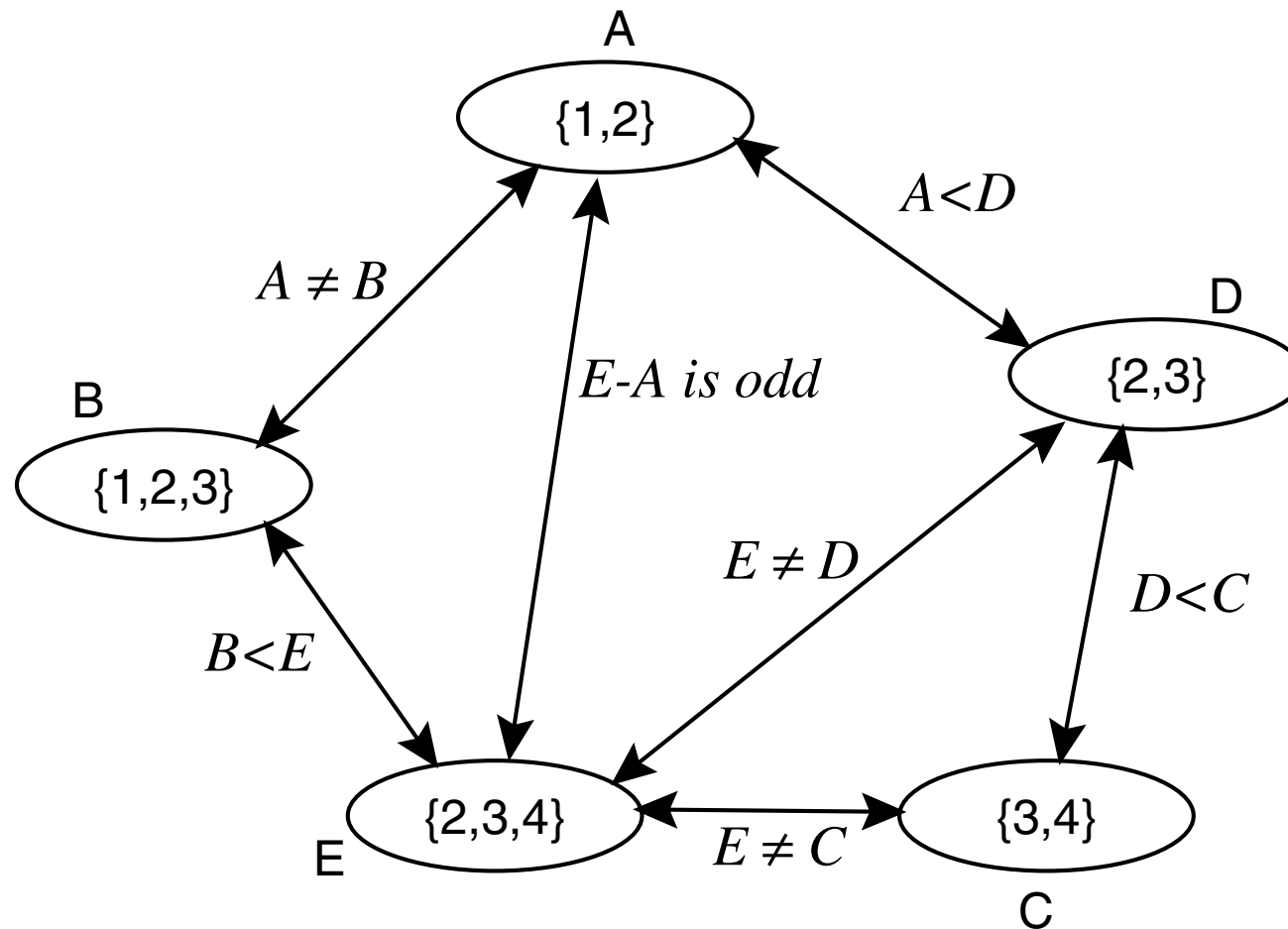
Intuition: the constraint constructed in line 5. summarizes the effect that all the constraints involving X have on variables other than X .

Example network



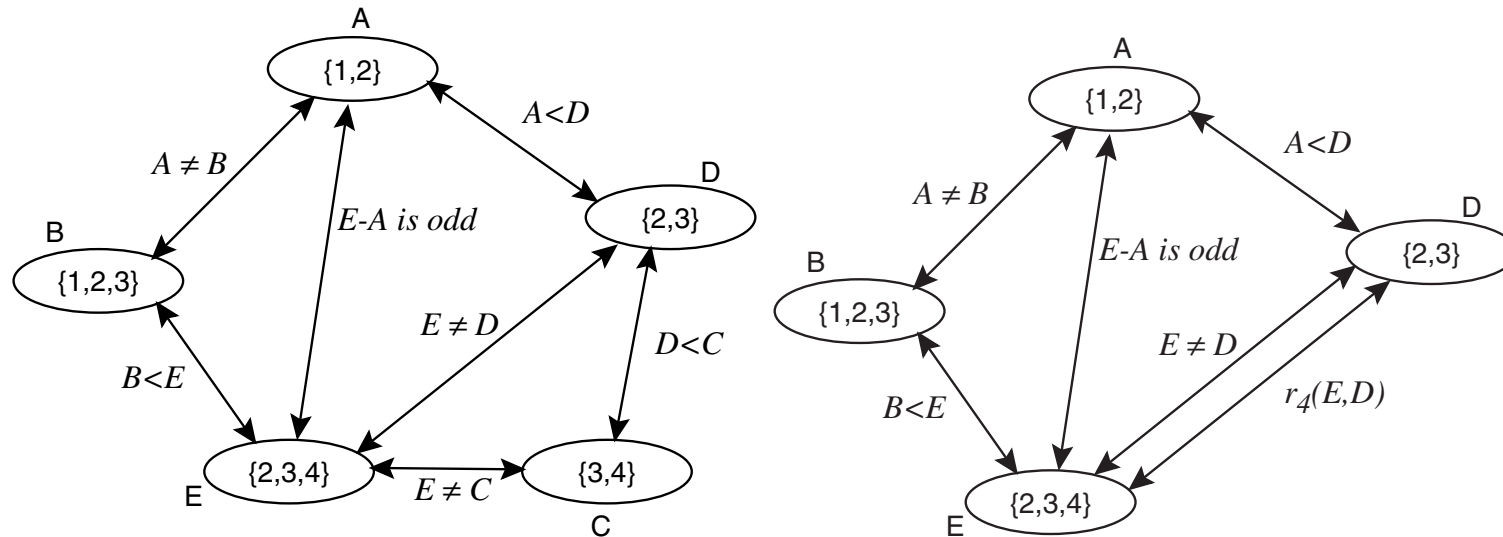
Example network

... now arc-consistent



Example network

Example: eliminating C



$r_1 : C \neq E$	C	E
	3	2
	3	4
	4	2
	4	3

$r_3 : r_1 \bowtie r_2$	C	D	E
	3	2	2
	3	2	4
	4	2	2
	4	2	3
	4	3	2
	4	3	3

$r_2 : C > D$	C	D
	3	2
	4	2
	4	3

$r_4 : \pi_{\{D,E\}} r_3$	D	E
	2	2
	2	3
	2	4
	3	2
	3	3

VE Properties

- The algorithm is **complete** (if there is a solution, backtracking will find it).
- The CSP has a solution if and only if the final constraint table is non-empty
- The set of all solutions can be generated by joining the final constraint with the intermediate “summarizing” constraints generated in line 5.
- Algorithm operates on extensional constraint representations, therefore
 - constraints must not contain too many tuples (initial and constructed constraints)
- Worst case: VE is not more efficient than enumerating all possible worlds and checking whether they are solutions.

Reminder: The Big Picture

- Say γ is a constraint network with n variables and maximal domain size k . **To solve γ , k^n total assignments must be tested in the worst case.**
- **Inference:** One method to try to avoid, or at least ameliorate, this explosion in practice.
 - *Often, from an assignment to some variables, we can easily make inferences regarding other variables.*
- **Decomposition:** Another method to try to avoid, or at least ameliorate, this explosion in practice.
 - *Often, we can exploit the **structure** of a network to **decompose** it into smaller parts that are easier to solve.*

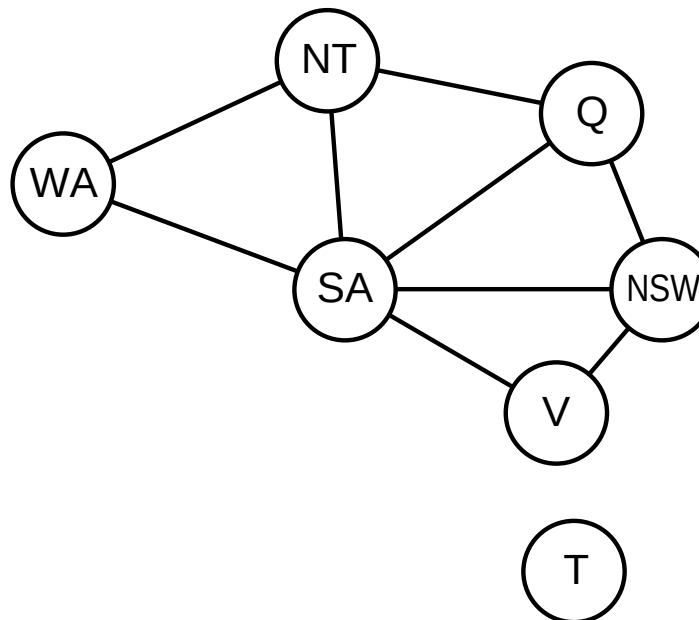
→ **What is “structure”, and how to “decompose”?**

For simplicity, in this section we assume that all constraints are binary.

“Structure”: Constraint Graphs

Definition (Constraint Graph). Let $\gamma = (V, D, C)$ be a constraint network. The **constraint graph** of γ is the undirected graph whose vertices are the variables V , and that has an arc $\{u, v\}$ if and only if $C_{uv} \in C$.

Example “Coloring Australia”:



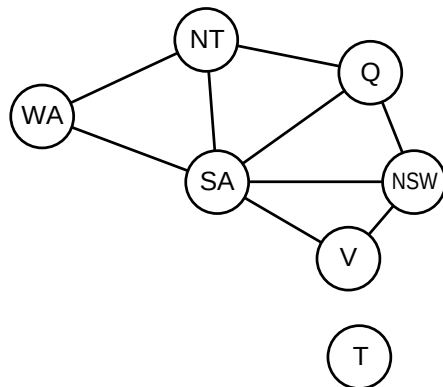
“Decomposition” 1.0: Disconnected Constraint Graphs

Theorem (Disconnected Constraint Graphs). Let $\gamma = (V, D, C)$ be a constraint network. Let a_i be a solution to each connected component V_i of the network's constraint graph. Then $a := \bigcup_i a_i$ is a solution to γ .

Proof. a satisfies all C_{uv} where u and v are inside the same connected component. The latter is the case for all C_{uv} .

→ If two parts of γ are not connected, then they are independent.

Examples:



→ Color Tasmania separately.

- γ with $n = 40$ variables, each domain size $k = 2$. Four separate connected components each of size 10.
- Worst-case without decomposition: 2^{40} .
Worst-case with decomposition: $4 * 2^{10}$. Gain: 2^{28} .

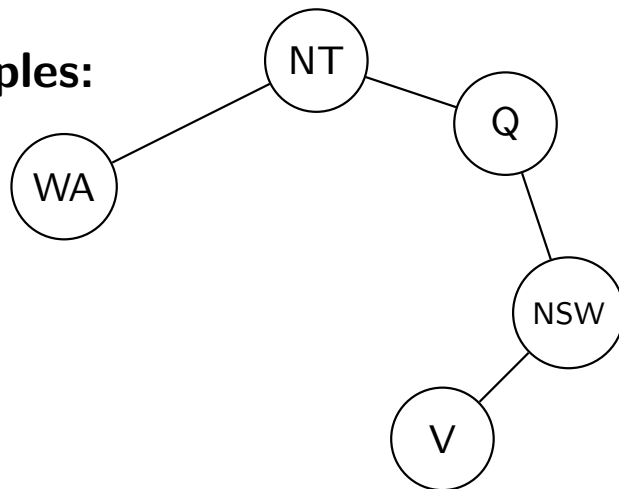
“Decomposition” 2.0: Acyclic Constraint Graphs

Theorem (Acyclic Constraint Graphs). Let $\gamma = (V, D, C)$ be a constraint network with n variables and K maximal domain size k , whose constraint graph is acyclic. Then we can find a solution for γ , or prove γ to be inconsistent, in time $O(nk^2 + n^2k)$.

Constraint networks with acyclic constraint graphs can be solved in low-order polynomial time.

How? → Either by using Variable Elimination or by the Acyclic CG algorithm in the additional material

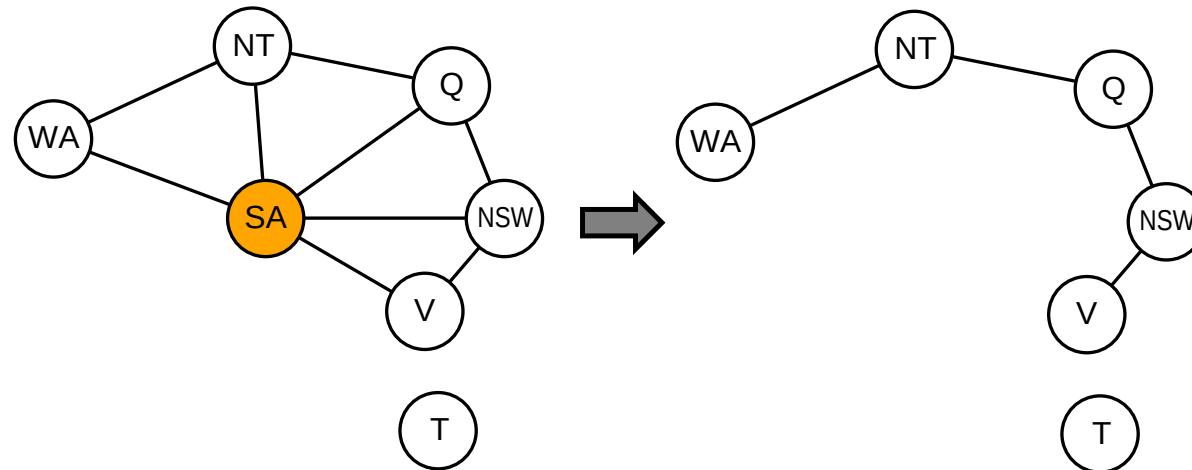
Examples:



- γ with $n = 40$ variables, each domain size $k = 2$. Acyclic constraint graph.
- Worst-case without decomposition: 2^{40} .
Worst-case with decomposition (see next slide): $40^2 + 40 + 40 * 2^2 + 40^2 * 2 < 2^{13}$.
Gain: $> 2^{27}$.

“Almost” Acyclic Constraint Graphs

Example “Coloring Australia”:



Cutset Conditioning: Idea

- Choose the variable order so that removing the first d variables renders the constraint graph acyclic.
 → **Then we won't have to search deeper than d , because:**
- Recursive call of backtracking on a s.t. the sub-graph of the constraint graph induced by $\{v \in V \mid a(v) \text{ is undefined}\}$ is acyclic:
 → **We can solve the remaining sub-problem with `AcyclicCG()`.**

Summary

- **Constraint Satisfaction Problems** γ consist of **variables**, associated with finite **domains**, and **constraints** which are binary relations specifying permissible value pairs.
- A **partial assignment** a maps some variables to values, a **total assignment** does so for all variables. a is **consistent** if it complies with all constraints. A consistent total assignment is a **solution**.
- **Backtracking** instantiates variables one-by-one, pruning inconsistent partial assignments.
- **Arc consistency** removes values that do not comply with any value still available at the other end of a constraint.
- **Variable Elimination** removes a variable by multiplying out all constraints that affect such variable.
- The **constraint graph** captures the dependencies between variables. Separate connected components can be solved independently. Networks with **acyclic constraint graphs** can be solved in low-order polynomial time.

Topics We Didn't Cover Here

- **Path consistency:** Generalizes arc consistency to multiple constraints.
- **Tree decomposition:** Instead of instantiating variables until the leaf nodes are trees, distribute the variables and constraints over sub-CSPs whose connections form a tree.
- **Backjumping:** Like backtracking, but with ability to back up *across several levels* (to a previous assignment identified to be responsible for failure).
- **No-Good Learning:** Inferring additional constraints based on information gathered during backtracking.
- **Local search:** In space of total (but not necessarily consistent) assignments.
- **Tractable CSP:** Classes of CSPs that can be solved in polynomial time.
- **Global Constraints:** Constraints over many/all variables, with associated specialized inference methods.
- **Constraint Optimization Problems (COP):** Utility function over solutions, need an optimal one.

Further Reading and Tools

- *Chapter 4: Reasoning with Constraints*, Sections 4.1–4.7.

There exist many tools that can solve constraint satisfaction problems:

- <https://developers.google.com/optimization/cp>
- <https://www.minizinc.org/>
- <http://picat-lang.org/>
- <http://www.constraintsolving.com/solvers>
- ...

You just specify the variables and constraints and automatically obtain a satisfying assignment!

The AcyclicCG Algorithm

Algorithm: **AcyclicCG**(γ)¹

- ① Obtain a directed tree from γ 's constraint graph, picking an arbitrary variable v as the root, and directing arcs outwards.
- ② Order the variables topologically, i.e., such that each vertex is ordered before its children; denote that order by $v_1, \dots, v_n$.
- ③ **for** $i := n, n - 1, \dots, 2$ **do**:
 - ① **Revise**($\gamma, v_{parent(i)}, v_i$).
 - ② **if** $D_{v_{parent(i)}} = \emptyset$ **then return** "inconsistent"

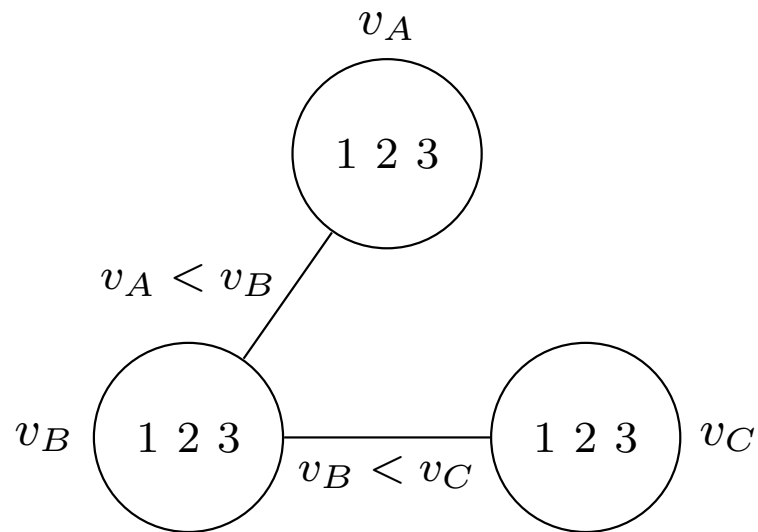
→ **Now, every variable is arc consistent relative to its children.**
- ④ Run BacktrackingWithInference with arc consistency, using the variable order $v_1, \dots, v_n$.
 → **This will find a solution without having to backtrack.** (Proof: Possible exercise for you)

→ Low polynomial runtime!

¹We assume that γ 's constraint graph is connected. If it is not, do the following for each connected component separately.

AcyclicCG(γ): Example

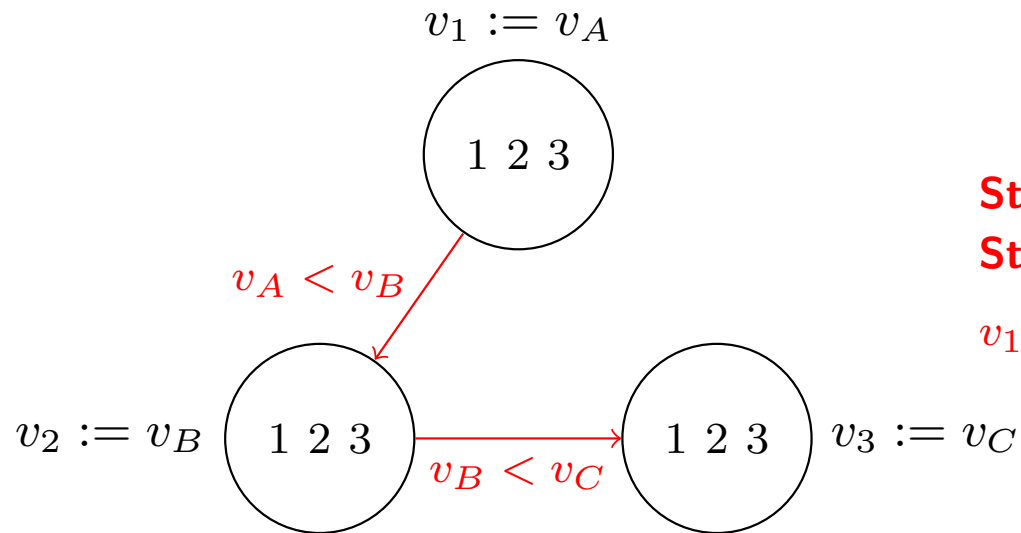
Example AcyclicCG() execution:



Input network γ

AcyclicCG(γ): Example

Example AcyclicCG() execution:

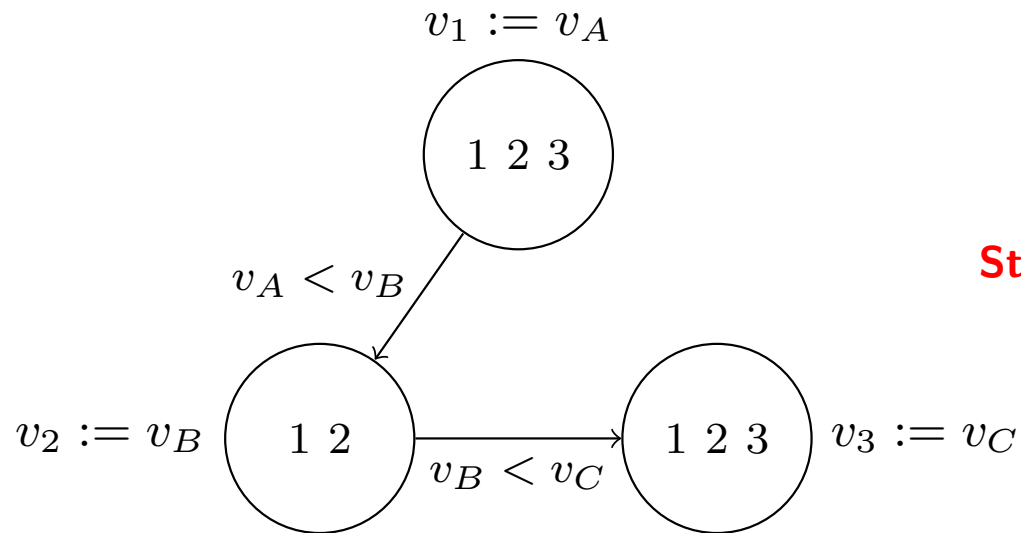


Step 1: Directed tree for root v_A .
Step 2: Order

$v_1 := v_A, v_2 := v_B, v_3 := v_C$

AcyclicCG(γ): Example

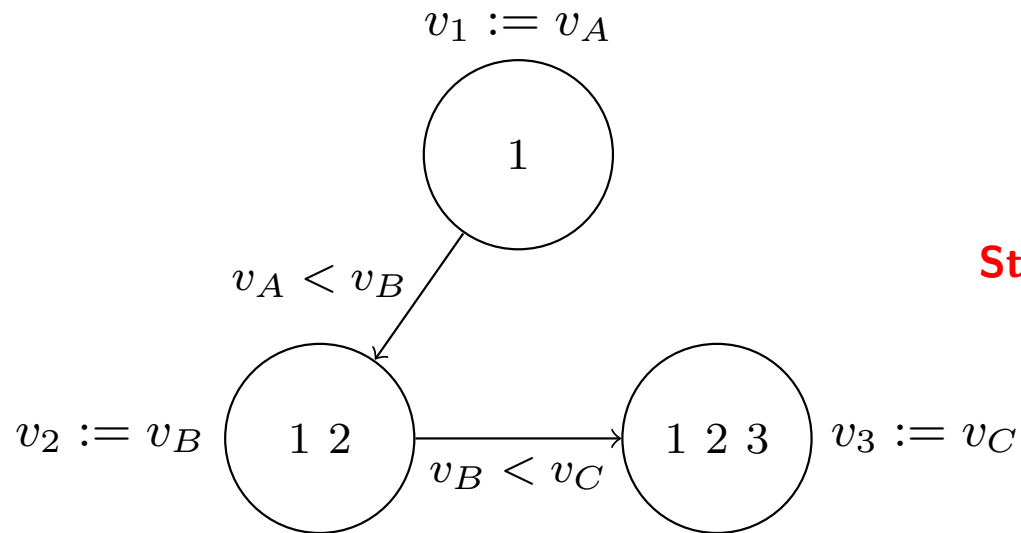
Example AcyclicCG() execution:



Step 3: After Revise(γ, v_2, v_3)

AcyclicCG(γ): Example

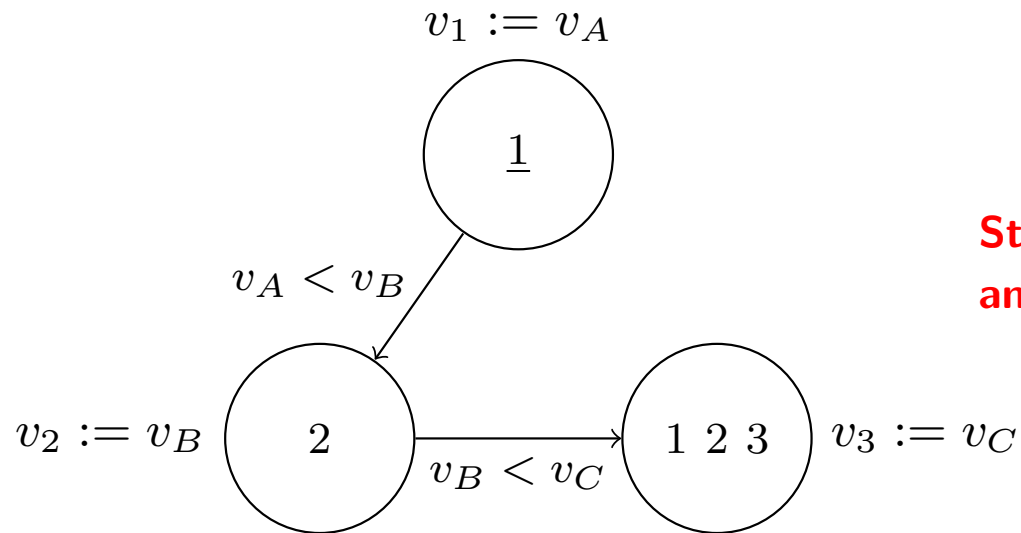
Example AcyclicCG() execution:



Step 3: After Revise(γ, v_1, v_2)

AcyclicCG(γ): Example

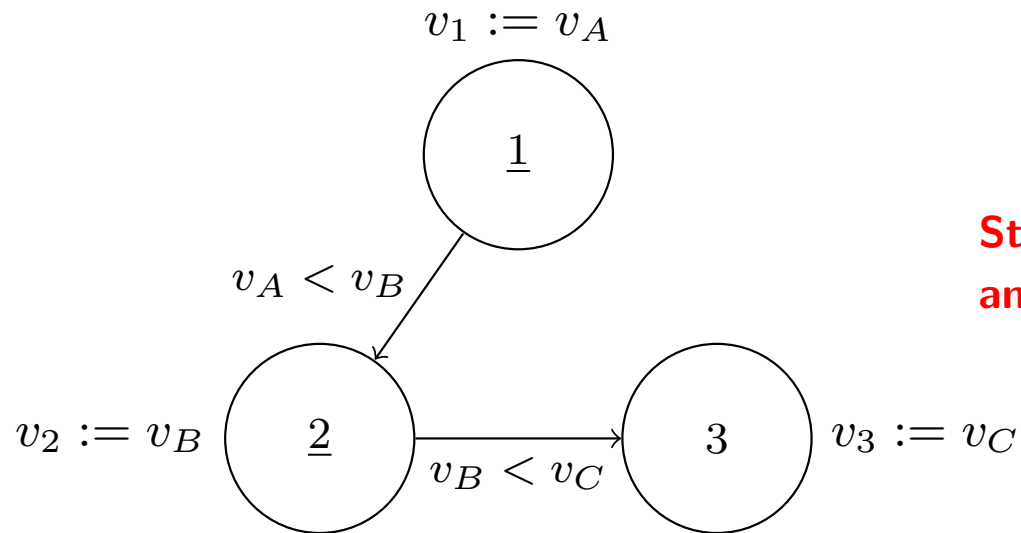
Example AcyclicCG() execution:



**Step 4: After $a(v_1) := 1$
and arc consistency**

AcyclicCG(γ): Example

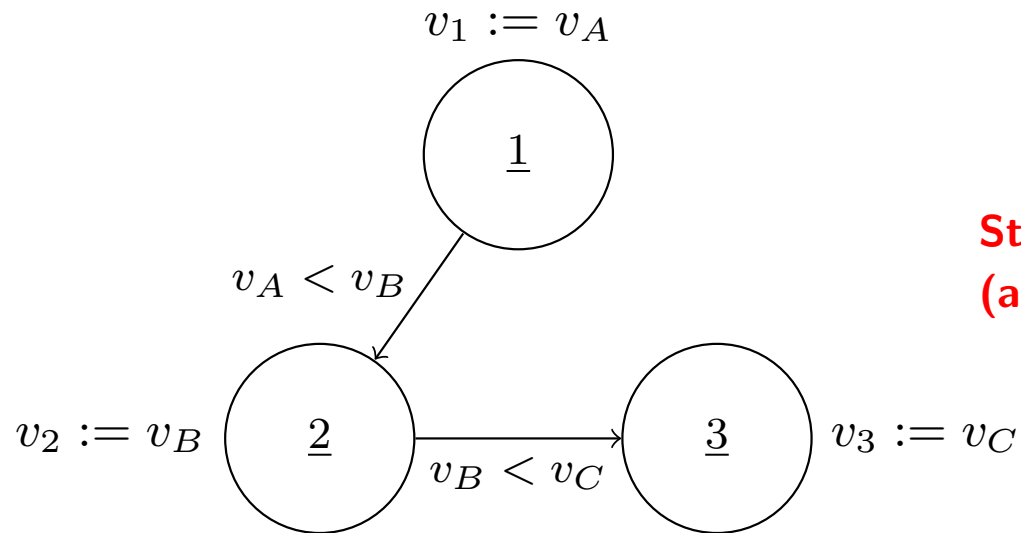
Example AcyclicCG() execution:



**Step 4: After $a(v_2) := 2$
and arc consistency**

AcyclicCG(γ): Example

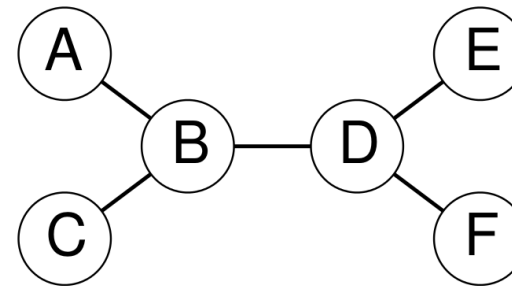
Example AcyclicCG() execution:



**Step 4: After $a(v_3) := 3$
(and arc consistency)**

Questionnaire

Constraint graph of γ :



Question!

How many different directed trees can we obtain/how many calls to `Revise()` are done for each?

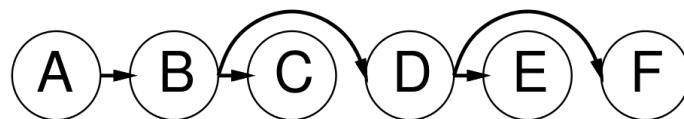
(A): 6 / 5

(B): 4 / 5

(C): 24 / 5

(D): 6 / Between 4 and 6

→ (A) is correct. Any vertex can be picked as the root, and once the root is picked the directed tree is unique. The number of calls to `Revise()` is the number of arcs in the tree and hence always is the number of arcs in the original constraint graph. Example:



→ `Revise(D, F)`, `Revise(D, E)`, `Revise(B, D)`,
`Revise(B, C)`, `Revise(A, B)`.

Systematic vs. Local Search

So far: all methods systematically explored the state space (possible worlds).

Problem: Time and space when search space is large.

Local Search approach:

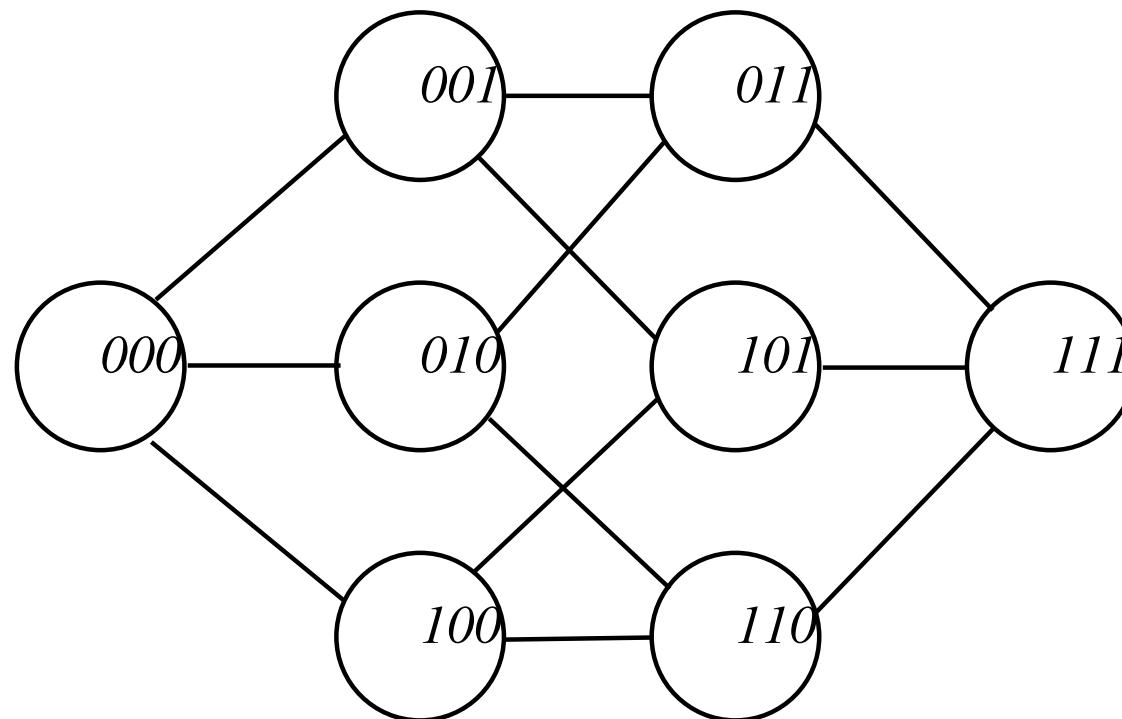
- explore state space without 'bookkeeping' (where have we been? what still needs to be explored?).
- no success/termination guarantees
- in practice, often the only thing that works

State Space Graph for CSP

(Another) state space graph representation for CSPs:

- Nodes are possible worlds
- Neighbors are possible worlds that differ in the value of exactly one variable

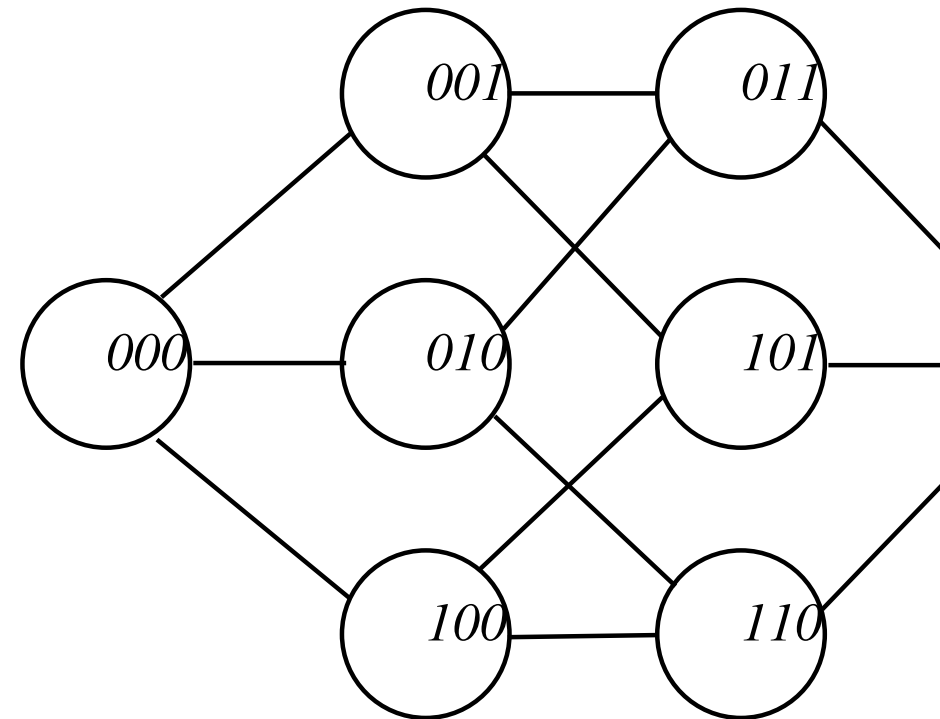
State space graph for 3 boolean variables:



Local Search Approach

Algorithm Outline

1. Select some node in state space graph as *current state*
2. **while** *current state* is not a solution
3. *current state* = some neighbor of *current state*



Random Search

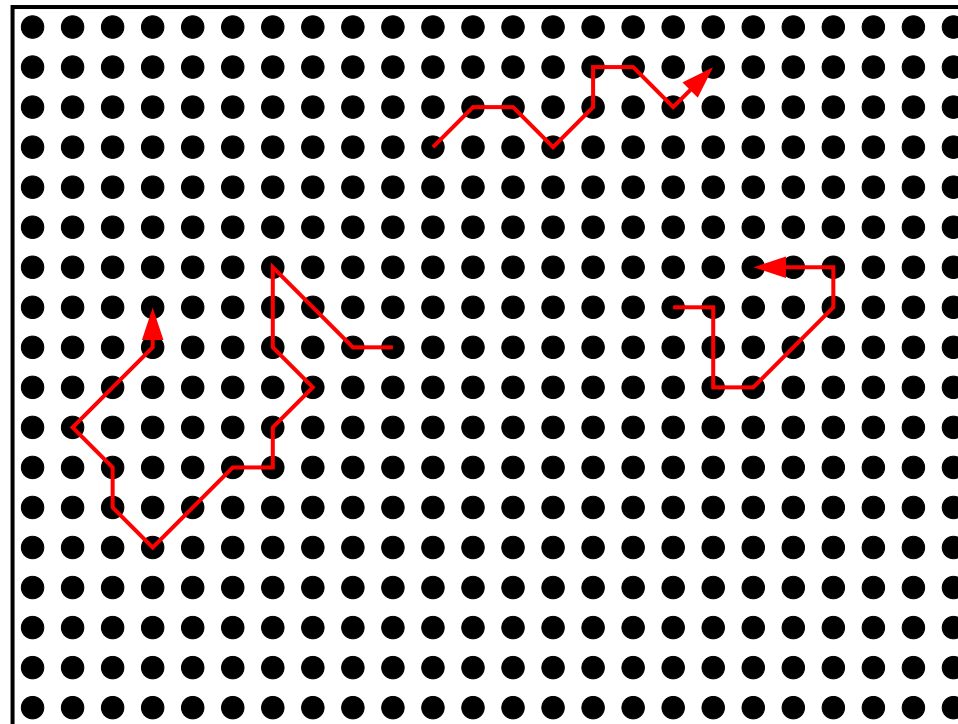
- Make choices in line 1. and 3. completely random
- “Random walk”
- Unlikely to find a solution if state space large with only few solutions

Greedy Search

Greedy Search or Hill Climbing:

- Use an *evaluation function* on states
- Example for evaluation function: number of constraints not satisfied by state
- Always choose neighbor with minimal evaluation function value
- Terminates when all neighbors have higher value than current state: current state is a **local minimum**.

Possible greedy search paths starting from different states:



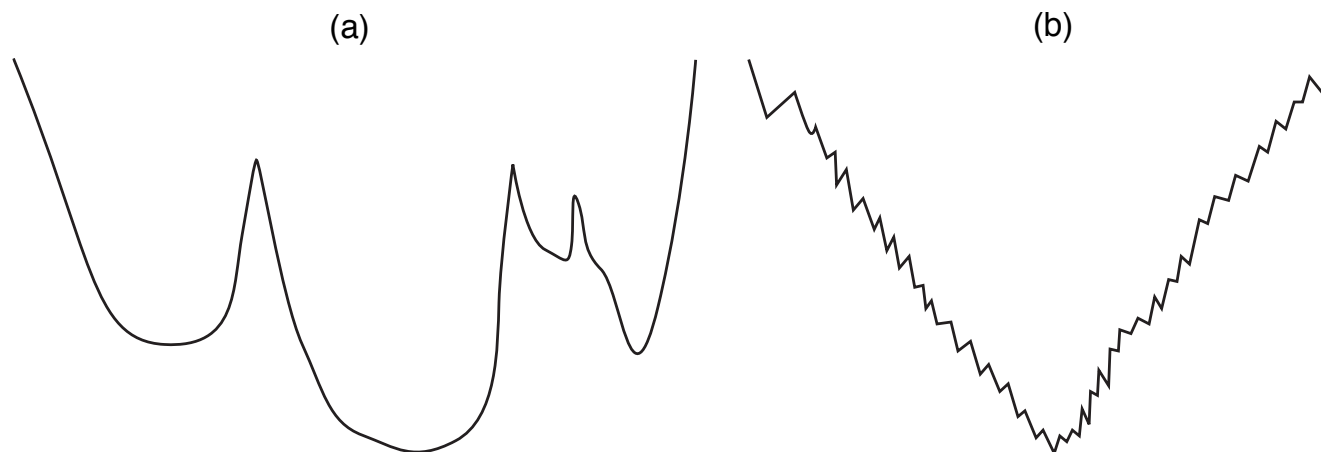
Escaping Local Minima

Problem

- Search terminates with local minimum of evaluation function. This may not be a solution to the CSP.

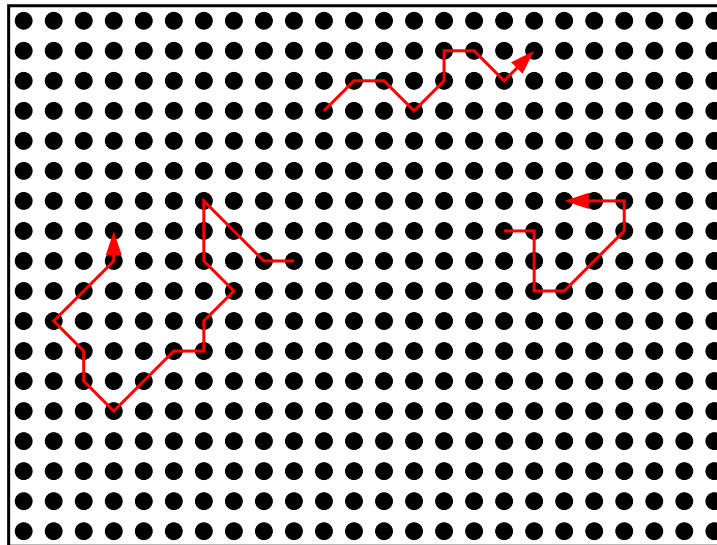
Solution Approaches

- Random restarts: repeat greedy search with several randomly chosen initial states
- Random moves: combine greedy moves with random steps
- **Example (a)**: Small number of random restarts will find global minimum
- **Example (b)**: Make random move when local minimum reached



Local search

- Maintain an assignment of a value to each variable.
- At each step, select a “neighbor” of the current assignment (e.g., one that improves some heuristic value).
- Stop when a satisfying assignment is found, or return the best assignment found.



Requires:

- What is a neighbor?
- Which neighbor should be selected?

Most improving step

Principle

Select the variable-value pair that gives the highest improvement.

Naive approach

- Linearly scan all variables and for each value of each variable determine the improvement (how many fewer constraints are violated).

Alternative

- Maintain a priority queue with variable-value pairs not part of the current assignment.
- $\text{Weight}\langle X, v \rangle = \text{eval}(\text{current assignment}) - \text{eval}(\text{current assignment but with } X = v).$
- If X is given a new value, update the weight of all pairs participating in a changed constraint.

Two-stage choice

Principle

- First: choose variable
- Second: choose state

Data structure

- Maintain priority queue of variables; weight is the number of participating conflicts.
- After selecting a variable, pick the value minimizes the number of conflicts.
- Update weights of variables that participate in a conflict that is changed.

Simulated annealing

Algorithm

- Pick a variable at random and a new value at random.
- If it is an improvement, adopt it.
- If it isn't an improvement, adopt it probabilistically depending on a temperature parameter, T .
 - With current assignment n and proposed assignment n' we move to n' with probability

$$e^{(h(n')-h(n))/T}$$

- Reduce the temperature.

Probability of accepting a change

Temperature	1-worse	2-worse	3-worse
10	0.91	0.81	0.74
1	0.37	0.14	0.05
0.25	0.02	0.0003	0.000005
0.1	0.00005	0	0

